

Образовательные треки ООО «Ред Софт»

КОД-09.02.06-1-2026

Сетевой и системный администратор

Модуль 2

Оглавление

Задание	3
Характеристики стенда	3
Топология	4
Таблица адресации	5
Анализ конфигурации стенда	5
1. Контроллер домена Samba DC	6
Задание	6
Реализация	6
2. Файловое хранилище на HQ-SRV	18
Задание	18
Реализация	18
3. Сервер сетевой файловой системы	22
Задание	22
Реализация	22
4. Служба сетевого времени	25
Задание	25
Реализация	25
5. Ansible	28
Задание	28
Реализация	28
6. Веб приложение в docker	32
Задание	32
Реализация	32
7. Веб приложение на HQ-SRV	39
Задание	39
Реализация	39
8. Статическая трансляция портов	44
Задание	44
Реализация	44
9. Обратный прокси-сервер на ISP	49
Задание	49
Реализация	49
10. Web-based аутентификация	52
Задание	52
Реализация	52
11. Установка Яндекс Браузер	54
Задание	54
Реализация	54
Приложение	56

Задание

Оценочные материалы

[КОД 09.02.06-1-2026 Сетевой и системный администратор](#)

[Оценочные материалы демонстрационного экзамена. Том 1](#)

[Приложение к образцу задания \(Том 1\)](#)

Характеристики стенда

ВМ	Оперативная память, Гб	ЦП, ядер	Накопитель, Гб	ОС
ISP	1	1	10	РЕД ОС сервер минимальный
HQ-RTR	1	1	10	
BR-RTR	1	1	10	
HQ-SRV	2	1	10	
BR-SRV	2	1	10	
HQ-CLI	3	2	20	РЕД ОС Рабочая станция
ИТОГО	10	7	70	

Топология

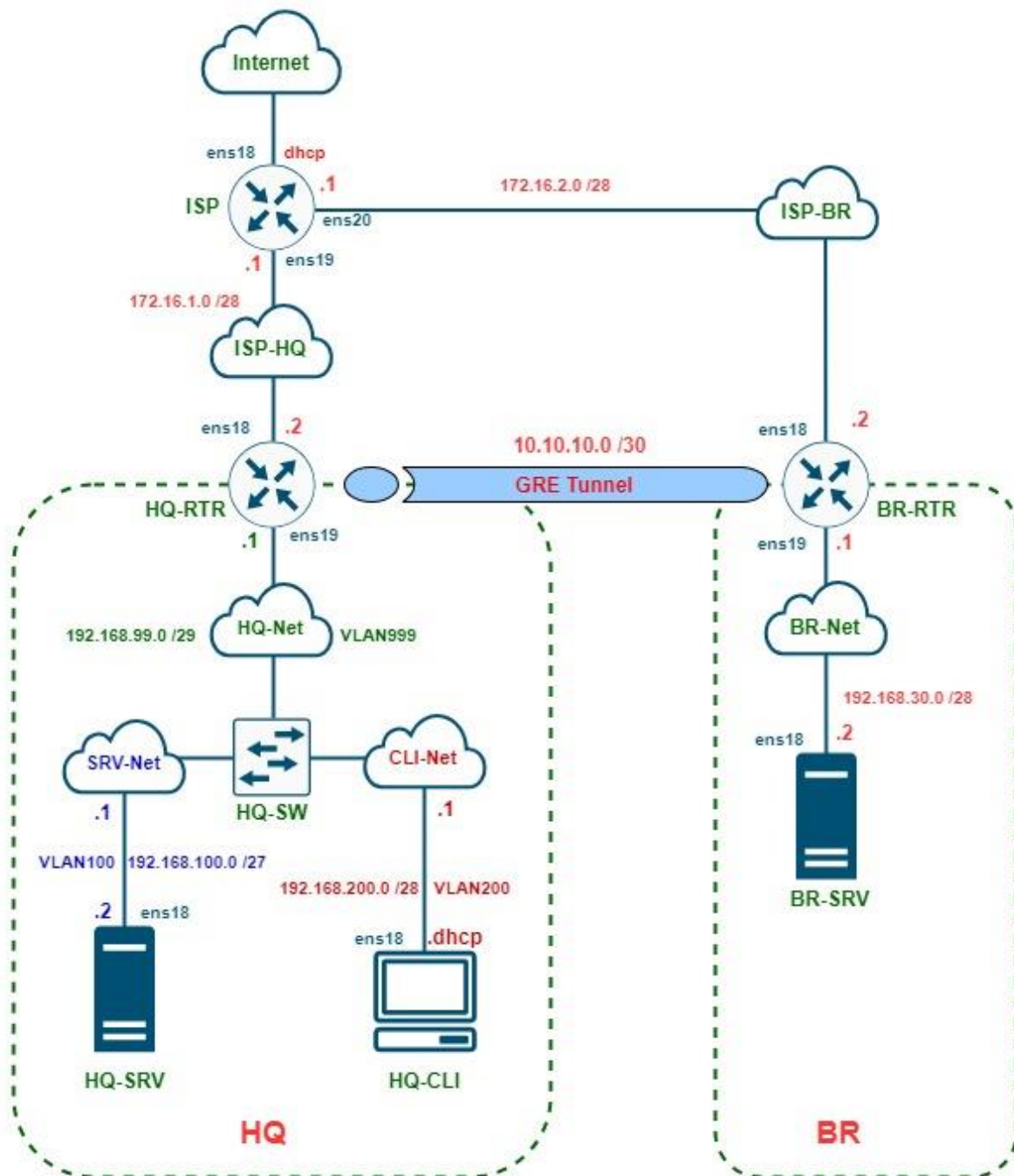


Рисунок 1. Топология Модуля 2

Таблица адресации

Имя устройства	Интерфейс	IP-адрес	Шлюз по умолчанию	Сеть
ISP	ens18	DHCP	–	Internet
	ens19	172.16.1.1/28	–	ISP-HQ
	ens20	172.16.2.1/28	–	ISP-BR
HQ-RTR	ens18	172.16.1.2/28	172.16.1.1	ISP-HQ
	VLAN100 (ens19)	192.168.100.1/27	–	SRV-Net
	VLAN200 (ens19)	192.168.200.1/28	–	CLI-Net
	VLAN999 (ens19)	192.168.99.1/29	–	HQ-Net
HQ-SRV	ens18	192.168.100.2/27	192.168.100.1	SRV-Net
HQ-CLI	ens18	DHCP	192.168.200.1	CLI-Net
BR-RTR	ens18	172.16.200.2/28	172.16.2.1	ISP-BR
	ens19	192.168.30.1/28	–	BR-Net
BR-SRV	ens18	192.168.30.2/28	192.168.30.1	BR-Net

Анализ конфигурации стенда

Стенд Модуля 2 преднастроен согласно Модуля 1

Проверка работоспособности стенда

1 Выполнить трассировку маршрута с BR-SRV:

```
BR-SRV -> HQ-SRV
traceroute 192.168.100.1
```

```
BR-SRV -> HQ-CLI
traceroute 192.168.200.3
```

2. Проверить работу служб:

```
HQ-SRV
# systemctl status named
```

```
HQ-RTR
# systemctl status dhcpd
```

1. Контроллер домена Samba DC

Задание

Настройте контроллер домена Samba DC на сервере BR-SRV:

- Имя домена au-team.irpo
- Введите в созданный домен машину HQ-CLI
- Создайте 5 пользователей для офиса HQ: имена пользователей формата hquserN^o (например hquser1, hquser2 и т.д.)
- Создайте группу hq, введите в группу созданных пользователей
- Убедитесь, что пользователи группы hq имеют право аутентифицироваться на HQ-CLI
- Пользователи группы hq должны иметь возможность повышать привилегии для выполнения ограниченного набора команд: cat, grep, id. Запускать другие команды с повышенными привилегиями пользователи группы права не имеют.

Реализация

1.1 Дополнительная настройка Bind на HQ-SRV

На HQ-SRV открываем конфигурационный файл DNS сервера `/etc/named.conf`

```
# nano /etc/named.conf
```

и дописываем строчку:

```
allow-transfer { 192.168.30.2; };
```

где:

`Allow-transfer` — директива в DNS-сервере BIND, которая указывает список серверов, которым разрешено брать зону с сервера. Обычно в этом параметре указывают вторичные (*slave*) серверы

`192.168.30.2` - IP адрес BR-SRV

```
GNU nano 8.8
named.conf

// Provided by Red Hat bind package to configure the ISC BIND named(8) DNS
// server as a caching only nameserver (as a localhost DNS resolver only).
// See /usr/share/doc/bind*/sample/ for example named configuration files.
//

options {
    listen-on port 53 { any; };
    listen-on-v6 port 53 { none; };
    directory "/var/named";
    dump-file "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    secroots-file "/var/named/data/named.secroots";
    recursing-file "/var/named/data/named.recursing";
    allow-query { any; };
    forwarders { 77.88.8.8; };
    allow-transfer { 192.168.38.2; };
    /*
    If you are building an AUTHORITY DNS server, do NOT enable r

```

Рисунок 2

Перезагружаем службу `named`

```
# systemctl restart named
```

1.2 Установка сервера SAMBA DC на BR-SRV

Вносим изменения в политики *SELinux*. на BR-SRV

Переводим *selinux* в режим уведомлений. Для этого изменяем содержимое конфигурационного файла:

```
# nano /etc/selinux/config
```

Заменяем текст *SELINUX=enforcing* на *SELINUX=permissive*.

Затем выполняем:

```
# setenforce 0
```

Устанавливаем Samba и Kerberos:

```
# dnf install samba* krb5* -y
```

Вносим изменения в настройки сетевого интерфейса *BR-SRV* в конфигурации IPv4

IP-адрес первого DNS-сервера – IP-адрес HQ-SRV

IP-адрес второго DNS-сервера – IP-адрес создаваемого контроллера домена (BR-SRV)

Указываем Домен поиска – *au-team.irpo*

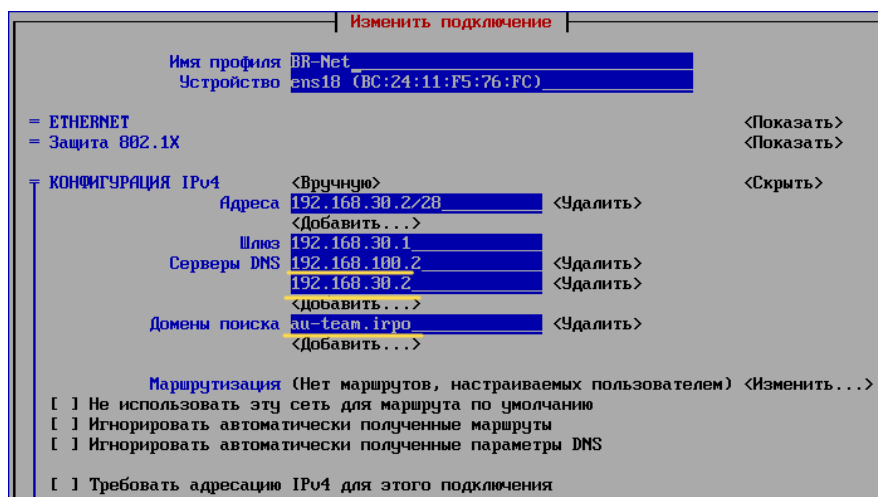


Рисунок 3

Для удобства и сокращения времени выполнения данного задания рекомендуется подключиться с HQ-CLI к BR-SRV по SSH. И произвести выполнения задания согласно [Базе Знаний РЕД ОС](#) (используем *ctrl+c ctrl+v*)

Отключаем DNS-службы *systemd-resolved* в файле */etc/systemd/resolved.conf*

```
# nano /etc/systemd/resolved.conf
```

Чтобы избежать конфликтов *systemd-resolved* с Samba DNS, нужно отключить *DNSStubListener*.

Устанавливаем параметр *DNSStubListener* в значение *no*, отключив прослушивание *systemd-resolved* на порту 53.


```
[Resolve]
# Some examples of DNS servers which may be used for DNS=
# Cloudflare: 1.1.1.1#cloudflare-dns.com 1.0.0.1#cloudfla
# Google: 8.8.8.8#dns.google 8.8.4.4#dns.google 2001:
# Quad9: 9.9.9.9#dns.quad9.net 149.112.112.112#dns.q
#DNS=
#FallbackDNS=
#Domains=
#DNSSEC=no
#DNSOverTLS=no
#MulticastDNS=no
#LLMNR=resolve
#Cache=yes
#CacheFromLocalhost=no
DNSStubListener=no
#DNSStubListenerExtra=
#ReadEtcHosts=yes
#ResolveUnicastSingleLabel=no
```

Рисунок 4

Перезапускаем `systemd-resolved` и `NetworkManager`

```
# systemctl restart systemd-resolved.service NetworkManager
```

Проверяем изменения в настройках

```
# cat /etc/resolv.conf
```

В выводе должен быть указан адрес, отличающийся от `127.0.0.53` и Домен поиска (`search`)

```
# See man:systemd-resolved.service(8) for
# operation for /etc/resolv.conf.

nameserver 192.168.100.2
nameserver 192.168.30.2
search au-team.irpo
[root@br-srv ~]# |
```

Рисунок 5

1.3 Создание домена под управлением Samba DC

Переименовываем файл `/etc/samba/smb.conf`, он будет создан в процессе выполнения команды `samba-tool`:

```
# mv /etc/samba/smb.conf /etc/samba/smb.conf.bak
```

Данный файла при запуске полуавтоматической конфигурации может вызвать ошибку.

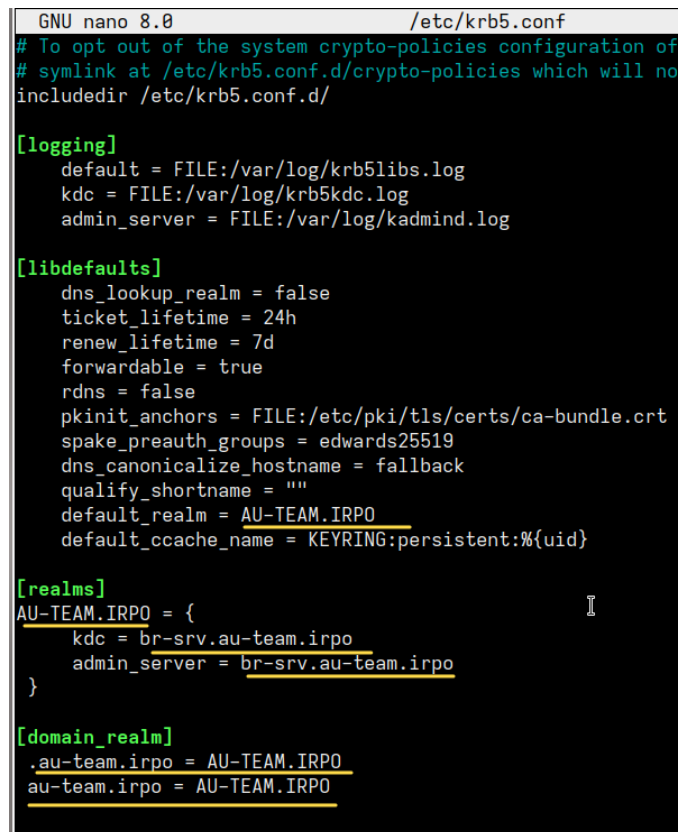
Создаем резервную копию конфигурационного файла `kerberos`, используемого по умолчанию

```
# cp /etc/krb5.conf /etc/krb5.conf.bak
```

Настраиваем конфигурацию Kerberos в `/etc/krb5.conf`

Данный файл приводим к следующему виду

```
# nano /etc/krb5.conf
```



```
GNU nano 8.0 /etc/krb5.conf
# To opt out of the system crypto-policies configuration of
# symlink at /etc/krb5.conf.d/crypto-policies which will no
includedir /etc/krb5.conf.d/

[logging]
default = FILE:/var/log/krb5libs.log
kdc = FILE:/var/log/krb5kdc.log
admin_server = FILE:/var/log/kadmind.log

[libdefaults]
dns_lookup_realm = false
ticket_lifetime = 24h
renew_lifetime = 7d
forwardable = true
rdns = false
pkinit_anchors = FILE:/etc/pki/tls/certs/ca-bundle.crt
spake_preauth_groups = edwards25519
dns_canonicalize_hostname = fallback
qualify_shortname = ""
default_realm = AU-TEAM.IRPO
default_ccache_name = KEYRING:persistent:%{uid}

[realms]
AU-TEAM.IRPO = {
    kdc = br-srv.au-team.irpo
    admin_server = br-srv.au-team.irpo
}

[domain_realm]
.au-team.irpo = AU-TEAM.IRPO
au-team.irpo = AU-TEAM.IRPO
```

Рисунок 6

Открываем файл `/etc/krb5.conf.d/crypto-policies`:

```
# nano /etc/krb5.conf.d/crypto-policies
```

и приводим его содержание к следующему виду:

```
[libdefaults]
default_tgs_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-
HMAC DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5

default_tkt_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-
HMAC DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5

preferred_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-HMAC
DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5
```

Можно скопировать с [Базы Знаний РЕД ОС](#)

```
GNU nano 8.0 /etc/krb5.conf.d/crypto-policies
[libdefaults]
default_tgs_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-HMAC DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5
default_tkt_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-HMAC DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5
preferred_enctypes = aes256-cts-hmac-sha1-96 aes128-cts-hmac-sha1-96 RC4-HMAC DES-CBC-CRC DES3-CBC-SHA1 DES-CBC-MD5
```

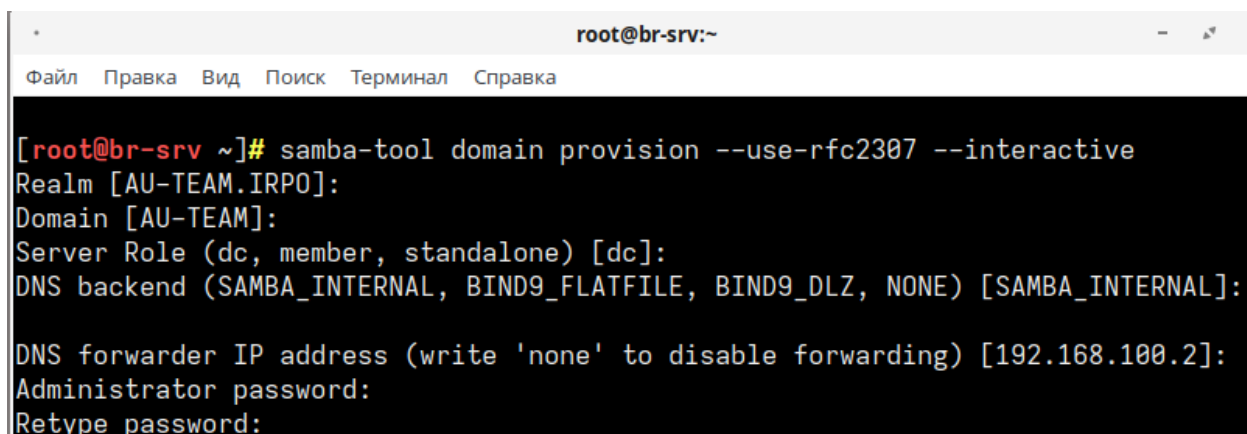
Рисунок 7

Запускаем полуавтоматическое конфигурирование сервера с помощью утилиты `samba-tool`.

Конфигурирование в интерактивном режиме выполняется командой:

```
# samba-tool domain provision --use-rfc2307 --interactive
```

Проверяем, если автоматически подставленные значения верные, ждем Enter и вводим пароль Администратора.



```
root@br-srv:~
Файл  Правка  Вид  Поиск  Терминал  Справка

[root@br-srv ~]# samba-tool domain provision --use-rfc2307 --interactive
Realm [AU-TEAM.IRPO]:
Domain [AU-TEAM]:
Server Role (dc, member, standalone) [dc]:
DNS backend (SAMBA_INTERNAL, BIND9_FLATFILE, BIND9_DLZ, NONE) [SAMBA_INTERNAL]:
DNS forwarder IP address (write 'none' to disable forwarding) [192.168.100.2]:
Administrator password:
Retype password:
```

Рисунок 8

Запускаем и добавляем в автозагрузку службу samba:

```
# systemctl enable --now samba
```

Проверяем статус службы:

```
# systemctl status samba
```

Проверяем работу домена:

```
# samba-tool domain info 127.0.0.1
# samba-tool domain info 192.168.30.2
```

```
[root@br-srv ~]#  
[root@br-srv ~]# samba-tool domain info 127.0.0.1  
Forest           : au-team.irpo  
Domain           : au-team.irpo  
Netbios domain   : AU-TEAM  
DC name          : br-srv.au-team.irpo  
DC netbios name  : BR-SRV  
Server site      : Default-First-Site-Name  
Client site      : Default-First-Site-Name  
[root@br-srv ~]#  
[root@br-srv ~]# samba-tool domain info 192.168.30.2  
Forest           : au-team.irpo  
Domain           : au-team.irpo  
Netbios domain   : AU-TEAM  
DC name          : br-srv.au-team.irpo  
DC netbios name  : BR-SRV  
Server site      : Default-First-Site-Name  
Client site      : Default-First-Site-Name  
[root@br-srv ~]#
```

Рисунок 9

1.4 Добавление пользователей и группы

Создаем 5 пользователей с помощью *samba-tool*:

```
# samba-tool user add hquser1 P@ssw0rd  
# samba-tool user add hquser2 P@ssw0rd  
# samba-tool user add hquser3 P@ssw0rd  
# samba-tool user add hquser4 P@ssw0rd  
# samba-tool user add hquser5 P@ssw0rd
```

Создаем группу *hq* и добавляем туда созданных пользователей:

```
# samba-tool group add hq
```

```
# samba-tool group addmembers hq hquser1,hquser2,hquser3,hquser4,hquser5
```

Вывод списка пользователей Samba DC:

```
# samba-tool user list
```

Вывод пользователей группы:

```
# samba-tool group listmembers hq
```

```
[root@br-srv ~]# samba-tool user list
hquser3
hquser4
krbtgt
hquser1
hquser5
Administrator
hquser2
Guest
[root@br-srv ~]#
[root@br-srv ~]# samba-tool group listmembers hq
hquser2
hquser3
hquser5
hquser1
hquser4
[root@br-srv ~]#
```

Рисунок 10

1.5 Ввод клиента HQ-CLI в домен

В настройке сетевого интерфейса HQ-CLI (*VLAN200*) добавляем Дополнительные серверы DNS – прописав IP адрес *BR-SRV*

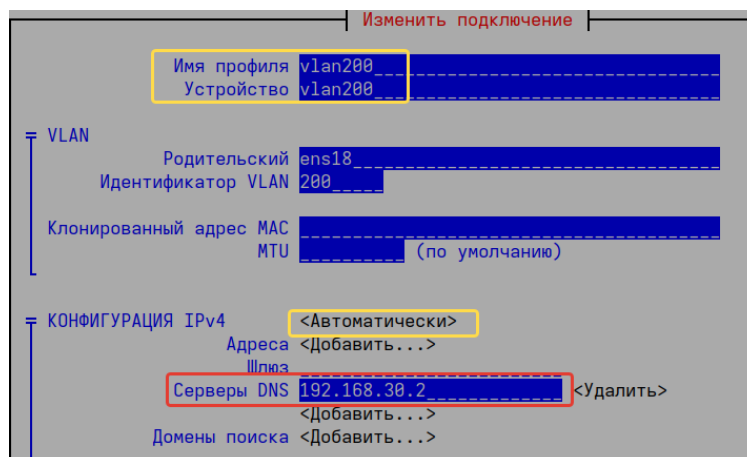
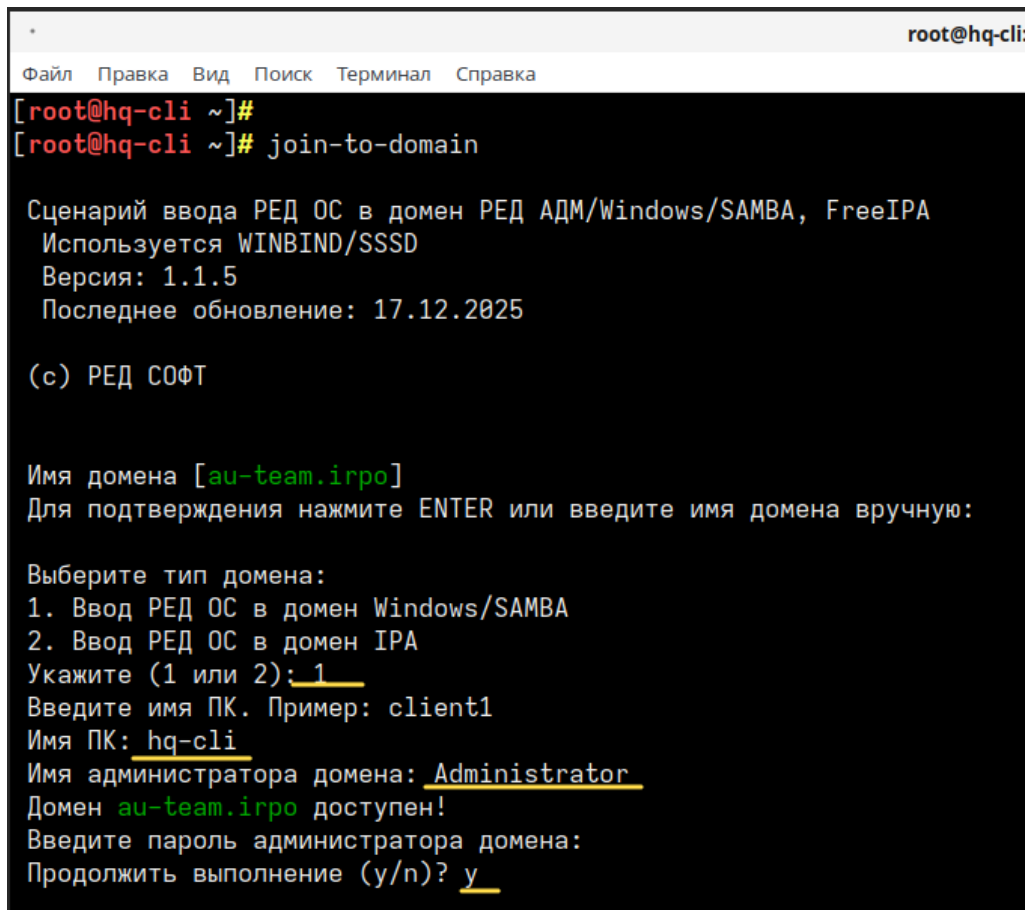


Рисунок 11

В терминале HQ-CLI запускаем скрипт `join-to-domain.sh` (или `join-to-domain`)



```
* root@hq-cli:
Файл Правка Вид Поиск Терминал Справка
[root@hq-cli ~]#
[root@hq-cli ~]# join-to-domain

Сценарий ввода РЕД ОС в домен РЕД АДМ/Windows/SAMBA, FreeIPA
Используется WINBIND/SSSD
Версия: 1.1.5
Последнее обновление: 17.12.2025

(с) РЕД СОФТ

Имя домена [au-team.irpo]
Для подтверждения нажмите ENTER или введите имя домена вручную:

Выберите тип домена:
1. Ввод РЕД ОС в домен Windows/SAMBA
2. Ввод РЕД ОС в домен IPA
Укажите (1 или 2): 1
Введите имя ПК. Пример: client1
Имя ПК: hq-cli
Имя администратора домена: Administrator
Домен au-team.irpo доступен!
Введите пароль администратора домена:
Продолжить выполнение (y/n)? y
```

Рисунок 12

Вводим *пароль*, который вводили при настройке домена через `samba-tool`

Перезагружаем HQ-CLI и входим под доменным пользователем.

Проверка

Выполняем на `BR-SRV`

Пример получения списка имен (SAM) компьютеров:

```
# samba-tool computer list
```

Пример получения списка уникальных составных имен компьютеров (DN):

```
# samba-tool computer list --full-dn
```

Получение полного списка учетных записей в домене:

```
# samba-tool user list
```

Пример получения списка уникальных составных имен пользователей (DN):

```
# samba-tool user list --full-dn
```

1.6 Привилегии для выполнения набора команд

Настройка производится на HQ-CLI

Вариант 1

Создаем файл `/etc/sudoers.d/hq` на HQ-CLI

```
# nano /etc/sudoers.d/hq
```

В данном файле создаем записи прав `sudo` для пользователей группы `hq`:

Повышение привилегий с вводом пароля

```
%hq ALL=(ALL) /usr/bin/cat, /usr/bin/grep, /usr/bin/id
```

Повышения привилегий без ввода пароля

```
%hq ALL=(ALL) NOPASSWD:/usr/bin/cat,/usr/bin/grep,/usr/bin/id
```

где

`/usr/bin/cat, /usr/bin/grep, /usr/bin/id` - полный путь к запускаемым командам

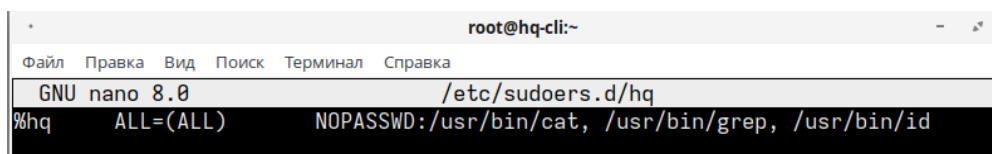


Рисунок 13

Вариант 2

Настройка пользователей группы *hq* повышать привилегии для выполнения ограниченного набора команд производится добавление записи для прав *sudo* в файле */etc/sudoers*:

```
# visudo
```

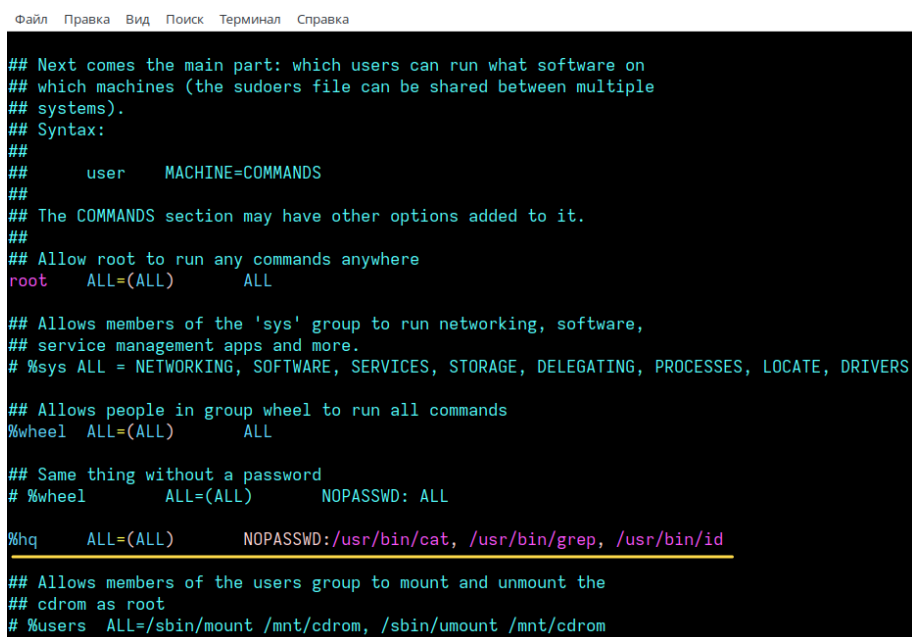
Вносим в конфигурационный файл следующие строки.

Повышение привилегий с вводом пароля:

```
%hq ALL=(ALL) /usr/bin/cat, /usr/bin/grep, /usr/bin/id
```

Повышения привилегий без ввода пароля:

```
%hq ALL=(ALL) NOPASSWD:/usr/bin/cat,/usr/bin/grep,/usr/bin/id
```



```
## Next comes the main part: which users can run what software on
## which machines (the sudoers file can be shared between multiple
## systems).
## Syntax:
##
##     user    MACHINE=COMMANDS
##
## The COMMANDS section may have other options added to it.
##
## Allow root to run any commands anywhere
root    ALL=(ALL)        ALL

## Allows members of the 'sys' group to run networking, software,
## service management apps and more.
# %sys ALL = NETWORKING, SOFTWARE, SERVICES, STORAGE, DELEGATING, PROCESSES, LOCATE, DRIVERS

## Allows people in group wheel to run all commands
%wheel  ALL=(ALL)        ALL

## Same thing without a password
# %wheel    ALL=(ALL)        NOPASSWD: ALL

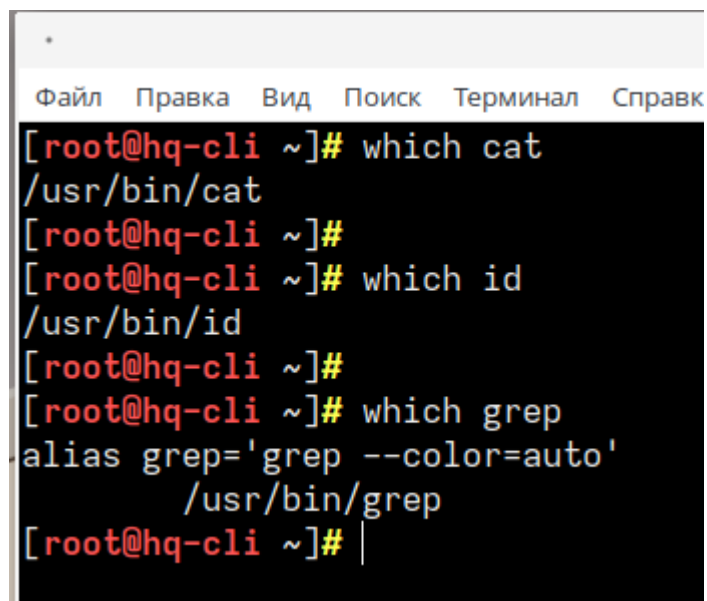
%hq     ALL=(ALL)        NOPASSWD:/usr/bin/cat, /usr/bin/grep, /usr/bin/id

## Allows members of the users group to mount and unmount the
## cdrom as root
# %users    ALL=/sbin/mount /mnt/cdrom, /sbin/umount /mnt/cdrom
```

Рисунок 14

Нахождение полного пути к команде

Which в Linux — это утилита для нахождения местоположения исполняемых файлов. Она анализирует пути, указанные в переменные окружения *PATH*, и возвращает путь к первому найденному файлу, соответствующему запрошенному имени.

A screenshot of a terminal window with a black background and red and yellow text. The window has a menu bar with "Файл", "Правка", "Вид", "Поиск", "Терминал", and "Справк". The terminal shows a root user at a prompt [root@hq-cli ~]# running the command "which cat", which returns "/usr/bin/cat". Then, the user runs "which id", returning "/usr/bin/id". Finally, the user runs "which grep", which returns "alias grep='grep --color=auto'" and "/usr/bin/grep". The prompt is followed by a vertical bar character "|".

```
[root@hq-cli ~]# which cat
/usr/bin/cat
[root@hq-cli ~]#
[root@hq-cli ~]# which id
/usr/bin/id
[root@hq-cli ~]#
[root@hq-cli ~]# which grep
alias grep='grep --color=auto'
/usr/bin/grep
[root@hq-cli ~]# |
```

Рисунок 15

2. Файловое хранилище на HQ-SRV

Задание

Сконфигурируйте файловое хранилище на сервере HQ-SRV:

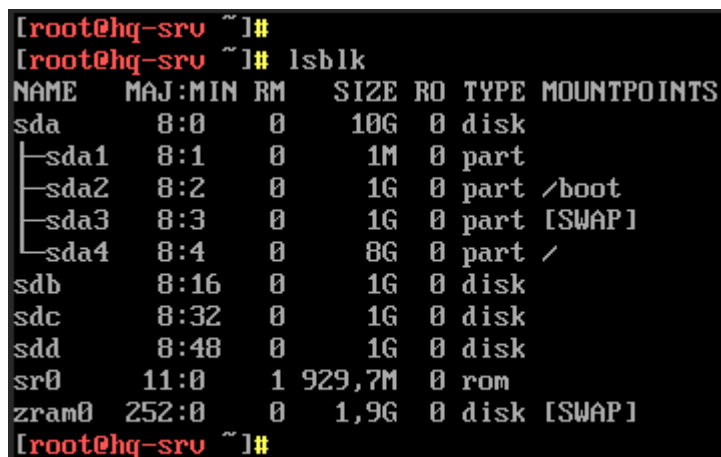
- При помощи двух подключенных к серверу дополнительных дисков размером 1 Гб сконфигурируйте дисковый массив уровня 0
- Имя устройства – md0, при необходимости конфигурация массива размещается в файле /etc/mdadm.conf
- Создайте раздел, отформатируйте раздел, в качестве файловой системы используйте ext4
- Обеспечьте автоматическое монтирование в папку /raid

Реализация

2.1 Создание программного RAID0

Выводим список подключенных дисков и их имена:

```
# lsblk
```



```
[root@hq-srv ~]#  
[root@hq-srv ~]# lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS  
sda           8:0    0   10G  0 disk  
├─sda1         8:1    0    1M  0 part  
├─sda2         8:2    0    1G  0 part /boot  
├─sda3         8:3    0    1G  0 part [SWAP]  
└─sda4         8:4    0    8G  0 part /  
sdb           8:16   0    1G  0 disk  
sdc           8:32   0    1G  0 disk  
sdd           8:48   0    1G  0 disk  
sr0          11:0    1 929,7M  0 rom  
zram0        252:0    0    1,9G  0 disk [SWAP]  
[root@hq-srv ~]# _
```

Рисунок 16

Для сборки массива применяем следующую команду:

```
# mdadm --create --verbose /dev/md0 -l 0 -n 2 /dev/sdb /dev/sdc
```

где:

/dev/md0 — устройство RAID, которое появится после сборки;

-l 0 – уровень RAID;

-n 2 – количество дисков, из которых собирается массив;

`/dev/sdb /dev/sdc` – сборка выполняется из дисков `sdb`, `sdc`.

Вводим команду:

```
# lsblk
```

и видим информацию о том, что у дисков `sdb`, `sdc`, появился раздел `md0`:

```
[root@hq-srv ~]#
[root@hq-srv ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda         8:0    0   10G  0 disk
├─sda1      8:1    0    1M  0 part
├─sda2      8:2    0    1G  0 part /boot
├─sda3      8:3    0    1G  0 part [SWAP]
└─sda4      8:4    0    8G  0 part /
sdb         8:16   0    1G  0 disk
sdc         8:32   0    1G  0 disk
sdd         8:48   0    1G  0 disk
sr0        11:0    1 929,7M  0 rom
zram0      252:0    0    1,9G  0 disk [SWAP]
[root@hq-srv ~]# mdadm --create --verbose /dev/md0 -l 0 -n 2 /dev/sdb /dev/sdc
mdadm: chunk size defaults to 512K
mdadm: Defaulting to version 1.2 metadata
mdadm: array /dev/md0 started.
[root@hq-srv ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda         8:0    0   10G  0 disk
├─sda1      8:1    0    1M  0 part
├─sda2      8:2    0    1G  0 part /boot
├─sda3      8:3    0    1G  0 part [SWAP]
└─sda4      8:4    0    8G  0 part /
sdb         8:16   0    1G  0 disk
└─md0       9:0    0    2G  0 raid0
sdc         8:32   0    1G  0 disk
└─md0       9:0    0    2G  0 raid0
sr0        11:0    1 929,7M  0 rom
zram0      252:0    0    1,9G  0 disk [SWAP]
[root@hq-srv ~]#
```

Рисунок 17

Создание файла `mdadm.conf`

В файле `mdadm.conf` находится информация о RAID-массивах и компонентах, которые в них входят. Для его создания выполняем следующую команду.

Сохраняем конфигурацию массива:

```
# mdadm --detail --scan --verbose >> /etc/mdadm.conf
```

2.2 Создание файловой системы для массива

Когда массив создан его необходимо отформатировать и смонтировать в дереве каталогов системы.

Отформатируем раздел в *ext4*:

```
# mkfs.ext4 /dev/md0
```

В корне системы создадим директорию */raid*:

```
# mkdir /raid
```

Монтируем к RAID директорию */raid* командой:

```
# mount /dev/md0 /raid
```

Результат проверяем командой:

```
# lsblk -f
```

```
[root@hq-srv ~]# lsblk -f
```

NAME	FSTYPE	FSVER	LABEL	UUID	FSAVAIL	FSUSE%	MOUNTPOINTS
sda							
└─sda1							
└─sda2	ext4	1.0		a6041ab2-6f96-4dc6-91c4-51d6574eae3	643,1M	27%	/boot
└─sda3	swap	1		50b9bd5e-eeaf-4865-bba4-cfadd5d3868a			[SWAP]
└─sda4	ext4	1.0		1f36f08a-61db-4d94-8fe0-0bb4a07a797f	3,5G	50%	/
sdb	linux_raid_member	1.2	hq-srv.au-team.irpo:0	23029b9f-b8dd-f9f2-1a76-54e279ea6280			
└─md0	ext4	1.0		edaecf01-7653-4c10-b3b3-d9d28c2b34d5	1,8G	0%	/raid
sdс	linux_raid_member	1.2	hq-srv.au-team.irpo:0	23029b9f-b8dd-f9f2-1a76-54e279ea6280			
└─md0	ext4	1.0		edaecf01-7653-4c10-b3b3-d9d28c2b34d5	1,8G	0%	/raid
sr0	iso9660		Joliet Extension Additional.iso	1899-12-30-00-00-00-00			
zram0							[SWAP]

```
[root@hq-srv ~]# _
```

Рисунок 18

Чтобы разделы монтировались при загрузке системы, добавляем запись в *fstab*

Открываем *fstab*:

```
# nano /etc/fstab
```

И добавляем строку:

```
/dev/md0 /raid ext4 defaults 0 0
```

Проверяем примонтированные разделы:

```
# df -h
```

```
[root@hq-srv ~]#  
[root@hq-srv ~]# df -h  
Файловая система  Размер  Использовано  Дост  Использовано%  Смонтировано в  
devtmpfs          4,0M      0             4,0M      0%             /dev  
tmpfs             984M      8,0K          984M      1%             /dev/shm  
tmpfs             394M      3,0M          391M      1%             /run  
/dev/sda4         7,8G      4,0G          3,5G      54%            /  
tmpfs             984M      0             984M      0%             /tmp  
/dev/sda2         974M      264M          644M      30%            /boot  
tmpfs             179M      4,0K          179M      1%             /run/user/0  
/dev/md0          2,0G      24K           1,9G      1%             /raid  
[root@hq-srv ~]# _
```

Рисунок 19

3. Сервер сетевой файловой системы

Задание

Настройте сервер сетевой файловой системы (nfs) на HQ-SRV:

- В качестве папки общего доступа выберите `/raid/nfs`, доступ для чтения и записи исключительно для сети в сторону HQ-CLI
- На HQ-CLI настройте автомонтирование в папку `/mnt/nfs`
- Основные параметры сервера отметьте в отчёте

Реализация

3.1 Настройка серверной части

На *HQ-SRV* устанавливаем компонент для серверной части *NFS*:

Устанавливаем необходимые пакеты:

```
# dnf install nfs4-acl-tools nfs-utils -y
```

Создаем папку общего доступа `/raid/nfs`:

```
# mkdir /raid/nfs
```

Задаем права на общую папку:

```
# chmod -R 777 /raid/nfs
```

Открываем на редактирование файл `/etc/exports` который содержит информацию о каталогах, экспортируемых с сервера.:

```
# nano /etc/exports
```

Создаем шару из каталога `/raid/nfs`, которая будет доступна сети в сторону HQ-CLI `192.168.200.0/28`.

Добавим в него строку:

```
/raid/nfs 192.168.200.0/28(rw,no_root_squash)
```

где:

/raid/nfs – общая сетевая папка

192.168.200.0/28 – сеть в сторону HQ-CLI

rw – разрешить чтение и запись.

no_root_squash – отключение ограничения root прав.



Рисунок 20

Добавляем в автозагрузку и запускаем *NFS* сервер

```
# systemctl enable --now nfs-server.service
```

Выполняем экспорт данных (перечитаем конфигурационный файл */etc/exports*, чтобы сервер начал отдавать настроенную шару):

```
# exportfs -arv
```



Рисунок 21

3.2 Настройка клиента

На HQ-CLI устанавливаем (если не установлен) компонент для клиентской части NFS:

```
# dnf install nfs-utils
```

Создаем директорию для автомонтирования */mnt/nfs*:

```
# mkdir /mnt/nfs
```

Задаём права на директорию */mnt/nfs*:

```
# chmod -R 777 /mnt/nfs/*
```

Для автоматического монтирования на HQ-CLI после перезагрузки, используем *fstab*.

Открываем файл *fstab*:

```
# nano /etc/fstab
```

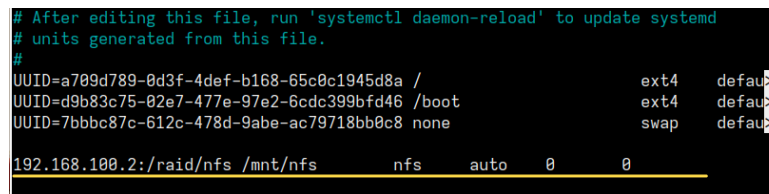
И добавляем строку:

```
192.168.100.2:/raid/nfs /mnt/nfs nfs auto 0 0
```

где

192.168.100.2 — адрес сервера NFS;

/mnt/nfs — каталог, куда будет примонтирована шара.



```
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#
UUID=a709d789-0d3f-4def-b168-65c0c1945d8a / ext4 defau>
UUID=d9b83c75-02e7-477e-97e2-6cdc399bfd46 /boot ext4 defau>
UUID=7bbbc87c-612c-478d-9abe-ac79718bb0c8 none swap defau>
192.168.100.2:/raid/nfs /mnt/nfs nfs auto 0 0
```

Рисунок 22

Производим монтирование общего ресурса:

```
# mount -av
```

Проверка монтирования

```
# df -h
```

Проверяем

Создаем на HQ-SRV файл

```
# touch /raid/nfs/test.txt
```

Проверяем на HQ-CLI

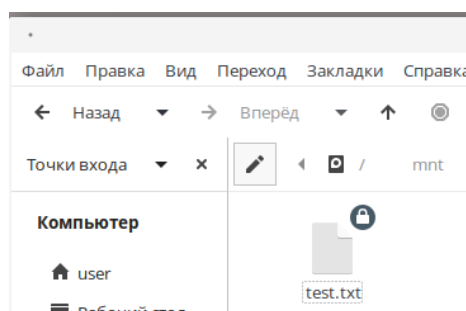


Рисунок 23

4. Служба сетевого времени

Задание

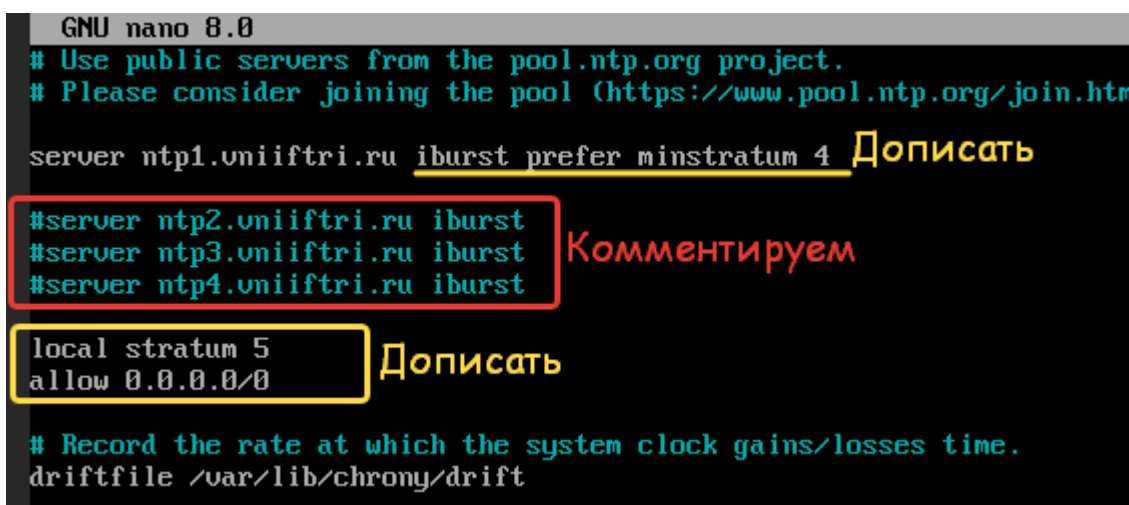
Настройте службу сетевого времени на базе сервиса chrony на маршрутизаторе ISP:

- Вышестоящий сервер ntp на маршрутизаторе ISP - на выбор участника
- Стратум сервера - 5
- В качестве клиентов ntp настройте: HQ-SRV, HQ-CLI, BR-RTR, BR-SRV

Реализация

4.1 Настройка NTP сервера на ISP

Открываем конфигурационный файл `/etc/chrony.conf` и вносим следующие изменения:



```
GNU nano 8.0
# Use public servers from the pool.ntp.org project.
# Please consider joining the pool (https://www.pool.ntp.org/join.htm

server ntp1.uniiftri.ru iburst prefer minstratum 4 Дописать
#server ntp2.uniiftri.ru iburst
#server ntp3.uniiftri.ru iburst
#server ntp4.uniiftri.ru iburst
local stratum 5
allow 0.0.0.0/0
# Record the rate at which the system clock gains/losses time.
driftfile /var/lib/chrony/drift
```

Рисунок 24

В параметр `allow` прописываем необходимые подсети; `0.0.0.0/0` – разрешаем всем

где:

`server` - сервер ntp, оставляем 1 из предложенных;
`iburst` — отправлять несколько пакетов (повышает точность);
`prefer` — указывает на предпочитаемый сервер;
`minstratum` - минимальный страт источника, чтобы избежать выбора;
`allow` — устройства с каких подсетей имеют возможность синхронизироваться с сервером;

local stratum — указываем stratum (стратум), т.е. слой.

Перезагружаем *chronyd*

```
# systemctl restart chronyd
```

Проверяем

```
[root@isp ~]#  
[root@isp ~]# chronyc sources  
MS Name/IP address         Stratum Poll Reach LastRx Last sample  
=====
```

MS Name/IP address	Stratum	Poll	Reach	LastRx	Last sample
* ntp1.uniiftri.ru	4	6	17	39	-82us[-7431ns] +/- 13ms

```
[root@isp ~]# chronyc sources_
```

Рисунок 25

4.2 Настройка клиентов

Офис HQ

На устройствах HQ-SRV, HQ-CLI вносим изменения в конфигурационный файл */etc/chrony.conf*

```
# nano /etc/chrony.conf
```

Комментируем *NTP-сервера* по умолчанию и указываем в качестве *NTP -сервера ISP*

```
GNU nano 8.0  
# Use public servers from the pool.ntp.org project.  
# Please consider joining the pool (https://www.pool.ntp.org/join.html).  
#server ntp1.uniiftri.ru iburst  
#server ntp2.uniiftri.ru iburst  
#server ntp3.uniiftri.ru iburst  
#server ntp4.uniiftri.ru iburst  
  
server 172.16.1.1 iburst prefer  
  
# Record the rate at which the system clock gains/losses time.  
driftfile /var/lib/chrony/drift
```

Рисунок 26

Перезагружаем *chronyd*:

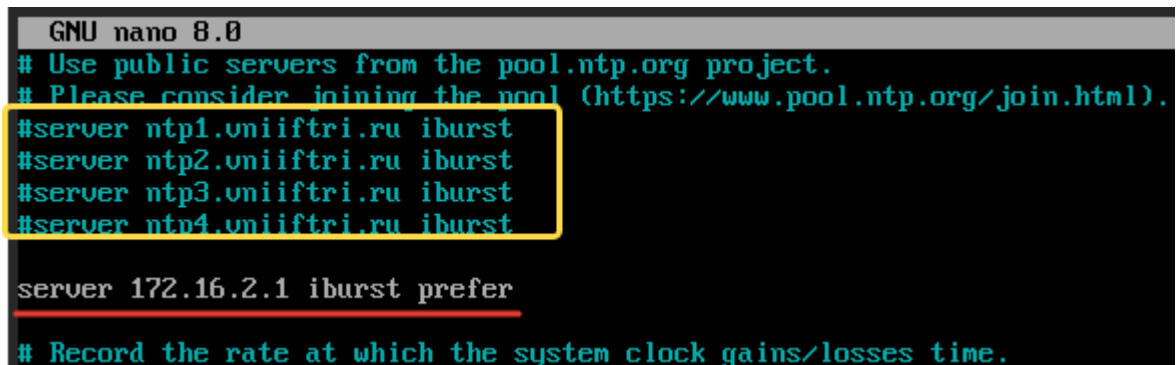
```
# systemctl restart chronyd
```

Офис BR

На устройствах BR-RTR, BR-SRV вносим изменения в конфигурационный файл `/etc/chrony.conf`

```
# nano /etc/chrony.conf
```

Комментируем *NTP-сервера по умолчанию* и указываем в качестве *NTP-сервера ISP*



```
GNU nano 8.0
# Use public servers from the pool.ntp.org project.
# Please consider joining the pool (https://www.pool.ntp.org/join.html).
#server ntp1.uniiftri.ru iburst
#server ntp2.uniiftri.ru iburst
#server ntp3.uniiftri.ru iburst
#server ntp4.uniiftri.ru iburst

server 172.16.2.1 iburst prefer

# Record the rate at which the system clock gains/losses time.
```

Рисунок 27

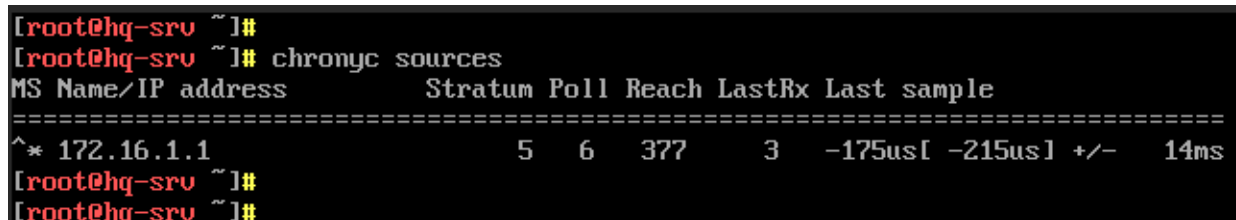
Перезагружаем `chronyd`:

```
# systemctl restart chronyd
```

4.3 Проверка

Проверить на клиенте можно командой

```
# chronyc sources
```



```
[root@hq-srv ~]#
[root@hq-srv ~]# chronyc sources
MS Name/IP address         Stratum Poll Reach LastRx Last sample
=====
^* 172.16.1.1                5      6   377     3  -175us[ -215us] +/-  14ms
[root@hq-srv ~]#
[root@hq-srv ~]#
```

Рисунок 28

5. Ansible

Задание

Сконфигурируйте ansible на сервере BR-SRV:

- Сформируйте файл инвентаря, в инвентарь должны входить HQ-SRV, HQ-CLI, HQ-RTR и BR-RTR
- Рабочий каталог ansible должен располагаться в /etc/ansible
- Все указанные машины должны без предупреждений и ошибок отвечать pong на команду ping в ansible посланную с BR-SRV.

Реализация

Устанавливаем *Ansible* и *sshpass* на BR-SRV:

```
# dnf install ansible sshpass -y
```

пакет *sshpass* – для подключения по SSH с аутентификацией по паролю, а не по ключам

Открываем файл инвентаря */etc/ansible/hosts*:

```
# nano /etc/ansible/hosts
```

Для создания файла инвентаря достаточно поместить в него список хостов (каждый новый хост – с новой строки). Имена хостов и IP-адреса взаимозаменяемы. В файле инвентаря распределяем устройства по группам (HQ и BR).

Сначала указывается название группы в квадратных скобках, затем перечисляются хосты.

Имена хостов прописываются в виде dns-имени хоста или ip.

После названия хоста указать параметры подключения к хосту в одну строку.

И в начале файла */etc/ansible/hosts* пишем:

```
[hq]
192.168.100.1 ansible_user=net_admin    ansible_password=P@ssw0rd
192.168.100.2 ansible_user=sshuser     ansible_password=P@ssw0rd ansible_port=2026
192.168.200.3 ansible_user=user       ansible_password=user

[br]
192.168.30.1 ansible_user=net_admin    ansible_password=P@ssw0rd
```

```
GNU nano 8.0 /etc/ansible
[!q]
192.168.100.1  ansible_user=net_admin  ansible_password=P@ssw0rd
192.168.100.2  ansible_user=sshuser      ansible_password=P@ssw0rd  ansible_port=2026
192.168.200.3  ansible_user=user         ansible_password=user

[!r]
192.168.30.1   ansible_user=net_admin  ansible_password=P@ssw0rd

# This is the default ansible 'hosts' file.
#
# It should live in /etc/ansible/hosts
```

Рисунок 29

где:

ansible_user и *ansible_password* - переменные, которые задают пользователя и пароль для определённой машины

ansible_port - параметр который указывает номер порта SSH (если порт не стандартный)

В файле конфигурации *ansible* */etc/ansible/ansible.cfg*, создаем группу *[defaults]* прописываем следующую строку:

```
host_key_checking = False
```

host_key_checking - параметр в Ansible, который отключает проверку ключей хоста при подключении по SSH. По умолчанию в Ansible включена эта проверка, которая защищает от подмены сервера.

```
GNU nano 8.0
# Since Ansible 2.12 (core):
# To generate an example config file (a "
#     $ ansible-config init --d
#
# Also you can now have a more complete f
# ansible-config init --disabled -t all >

# For previous versions of Ansible you can
# Note that this file was always incomple

# for example, for 2.9: https://github.com

[defaults]
host_key_checking = False
```

Рисунок 30

Запускаем модуль *ping* на всех хостах, перечисленных в файле инвентаря */etc/ansible/hosts*:

```
# ansible all -m ping
```

Может появиться предупреждение про обнаружение интерпретатора *Python*, на целевом хосте

```
[root@br-srv ~]# ansible all -m ping
[WARNING]: Platform linux on host 192.168.30.1 is using the discovered Python interpreter
which could change the meaning of that path. See https://docs.ansible.com/ansible/latest/reference_appendices/config.html#python-interpreter
192.168.30.1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
[WARNING]: Platform linux on host 192.168.100.1 is using the discovered Python interpreter
which could change the meaning of that path. See https://docs.ansible.com/ansible/latest/reference_appendices/config.html#python-interpreter
192.168.100.1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
```

Рисунок 31

Для управления поведением обнаружения в глобальном масштабе необходимо в файле конфигурации ansible `/etc/ansible/ansible.cfg` в разделе `[defaults]` прописать ключ `interpreter_python` с параметром `auto_silent`

```
GNU nano 8.0
# Since Ansible 2.12 (core):
# To generate an example config file (a "disabled" file) run:
# $ ansible-config init --disabled
#
# Also you can now have a more complete file by
# ansible-config init --disabled -t all > ansible.cfg
#
# For previous versions of Ansible you can check
# Note that this file was always incomplete and
# for example, for 2.9: https://github.com/ansible/ansible/blob/devel/examples/ansible.cfg
[defaults]
host_key_checking = False
interpreter_python=auto_silent
```

Рисунок 32

Запускаем команду `ping` на всех хостах:

```
# ansible all -m ping
```

```
[root@br-srv ~]# ansible all -m ping
192.168.30.1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
192.168.100.2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
192.168.100.1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
192.168.200.3 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.11"
  },
  "changed": false,
  "ping": "pong"
}
[root@br-srv ~]#
```

Рисунок 33

6. Веб приложение в docker

Задание

Разверните веб приложение в docker на сервере BR-SRV:

- Средствами docker должен создаваться стек контейнеров с веб приложением и базой данных
- Используйте образы `site_latest` и `mariadb_latest` располагающиеся в директории `docker` в образе `Additional.iso`
- Основной контейнер `testapp` должен называться `tespapp`
- Контейнер с базой данных должен называться `db`
- Импортируйте образы в `docker`, укажите в `yaml` файле параметры подключения к СУБД, имя БД - `testdb`, пользователь `test` с паролем `P@ssw0rd`, порт приложения `8080`, при необходимости другие параметры
- Приложение должно быть доступно для внешних подключений через порт `8080`

Реализация

6.1 Установка docker и docker-compose

Устанавливаем Docker и Docker Compose на BR-SRV:

```
# dnf install docker-ce docker-ce-cli docker-compose -y
```

После установки запускаем сервис контейнеризации `docker` и добавляем его в автозагрузку:

```
# systemctl enable docker --now
```

Проверка статуса запущенной службы:

```
# systemctl status docker
```

Смотрим подключен ли `cd-rom` с `Additional.iso`:

```
# lsblk
```



```
[root@br-srv ~]#
[root@br-srv ~]#
[root@br-srv ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda           8:0    0   12G  0 disk
├─sda1        8:1    0    1M  0 part
├─sda2        8:2    0    1G  0 part /boot
├─sda3        8:3    0   1.2G  0 part [SWAP]
└─sda4        8:4    0   9.8G  0 part /
sr0          11:0    1 929.7M  0 rom
zram0        252:0    0   1.9G  0 disk [SWAP]
[root@br-srv ~]#
```

Рисунок 34

6.2 Монтирование образов

Монтируем Additional.iso:

```
# mount /dev/sr0 /mnt/
```

Смотрим содержимое:

```
# ls -l /mnt/
```

```
# ls -l /mnt/docker
```

```
[root@br-srv ~]#
[root@br-srv ~]# mount /dev/sr0 /mnt/
mount: /mnt: WARNING: source write-protected, mounted read-only.
[root@br-srv ~]#
[root@br-srv ~]# ls -l /mnt/
итого 34
dr-xr-xr-x. 1 root root  332 дек 30  2155 docker
dr-xr-xr-x. 1 root root  150 дек 30  2155 playbook
-r-xr-xr-x. 1 root root 32527 окт 13  00:22 Users.csv
dr-xr-xr-x. 1 root root  220 дек 30  2155 web
[root@br-srv ~]#
[root@br-srv ~]# ls -l /mnt/docker/
итого 951936
-r-xr-xr-x. 1 root root 333014016 окт  8 09:17 mariadb_latest.tar
-r-xr-xr-x. 1 root root 282003968 окт  8 09:17 postgresql_latest.tar
-r-xr-xr-x. 1 root root   2716 окт  8 09:15 readme.txt
-r-xr-xr-x. 1 root root 359760896 окт  8 09:16 site_latest.tar
[root@br-srv ~]#
[root@br-srv ~]#
```

Рисунок 35

Импортируем образы *mariadb_latest* и *site_latest*:

```
# docker load < /mnt/docker/site_latest.tar
```

```
# docker load < /mnt/docker/mariadb_latest.tar
```

```
[root@br-srv ~]# docker load < /mnt/docker/site_latest.tar
fd2758d7a59e: Loading layer [=====>] 8.594MB/8.594MB
40b6a95a818e: Loading layer [=====>] 1.689MB/1.689MB
340d7d095707: Loading layer [=====>] 37.29MB/37.29MB
5a9854a10c2e: Loading layer [=====>] 5.12kB/5.12kB
b3ec3c3769f3: Loading layer [=====>] 1.536kB/1.536kB
12d275e5fc40: Loading layer [=====>] 176.4MB/176.4MB
43e94e56cf9e: Loading layer [=====>] 3.072kB/3.072kB
b697f445a0f2: Loading layer [=====>] 129.1MB/129.1MB
79ad03de2e0b: Loading layer [=====>] 842.0kB/842.0kB
Loaded image: site:latest
[root@br-srv ~]#
[root@br-srv ~]# docker load < /mnt/docker/mariadb_latest.tar
f862e1968e4b: Loading layer [=====>] 80.41MB/80.41MB
f034330494cf: Loading layer [=====>] 337.9kB/337.9kB
9c500283cbdf: Loading layer [=====>] 16.33MB/16.33MB
0111a34bc1cb: Loading layer [=====>] 1.536kB/1.536kB
1bbf6834d68a: Loading layer [=====>] 5.12kB/5.12kB
2be2f7b4f7a7: Loading layer [=====>] 235.6MB/235.6MB
e8ecae52c35a: Loading layer [=====>] 13.82kB/13.82kB
43e9e08bf0e1: Loading layer [=====>] 29.7kB/29.7kB
Loaded image: mariadb:latest
[root@br-srv ~]#
```

Рисунок 36

Выводим список локально сохранённых образов:

```
# docker image ls
```

```
[root@br-srv ~]#
[root@br-srv ~]# docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
site           latest    015b4b821098   4 months ago   353MB
mariadb        10.11     bc52d24721da   6 months ago   327MB
[root@br-srv ~]#
```

Рисунок 37

6.3 Создание файла для работы с контейнерами

Смотрим инструкцию от разработчиков в файле `readme.txt`:

```
# cat /mnt/docker/readme.txt
```

```
[root@br-srv ~]# cat /mnt/docker/readme.txt
Разработчик приложения testapp приветствует вас!

Наше приложение выполняет очень важную бизнес-задачу и поставляется в виде набора TAR-архивов с образами контейнеров для Docker.

site_latest.tar - основной контейнер веб-приложения, использующий порт tcp/8080

переменные для запуска
DB_TYPE - [mariadb|postgres]
DB_HOST - <IPv4 СЮБД>
DB_NAME - <название БД>
DB_PORT - <порт TCP подключения к БД>
DB_USER - <пользователь СЮБД>
DB_PASS - <пароль пользователя СЮБД>

mariadb_latest.tar - контейнер СЮБД MariaDB, использующий порт tcp/3306

переменные для запуска
MARIADB_DATABASE - <название БД>
MARIADB_USER - <пользователь СЮБД>
MARIADB_PASSWORD - <пароль пользователя СЮБД>
MARIADB_ROOT_PASSWORD/MARIADB_RANDOM_ROOT_PASSWORD - обязательные параметры

postgres_latest.tar - контейнер СЮБД PostgreSQL, использующий порт tcp/5432

переменные для запуска
POSTGRES_DB - <название БД>
POSTGRES_USER - <пользователь СЮБД>
POSTGRES_PASSWORD - <пароль пользователя СЮБД>

Все заявленные переменные являются обязательными для запуска и должны быть согласованы по значениям.

Проверка работоспособности приложения проводится путем заполнения 3 строк данными с последующим перезапуском узла. Записи должны сохраниться при повторном обращении к сайту. Так же, контроль может быть произведен путем проверки записи в БД вносимых через веб-интерфейс данных.

Данное приложение поставляется as is, разработчик не несёт никакой ответственности за возможные убытки с чьей-либо стороны, запрещает вносить изменения или коммитировать его исходный код где бы то ни было.

Использовать только для нужд сдачи ДЗ или подготовки к ДЗ.
[root@br-srv ~]#
```

Рисунок 38

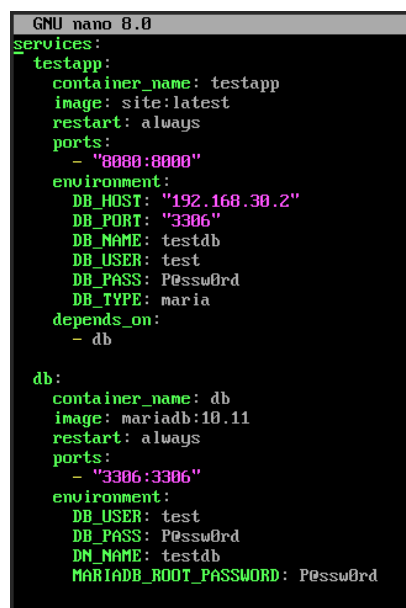
Создаем файл `web.yaml` описывающий стек контейнеров для развертывания:

```
# nano web.yaml
```

Со следующим содержимым:

```
services:
  testapp:
    container_name: testapp
    image: site:latest
    restart: always
    ports:
      - "8080:8000"
    environment:
      DB_HOST: "192.168.30.2"
      DB_PORT: "3306"
      DB_NAME: testdb
      DB_USER: test
      DB_PASS: P@ssw0rd
      DB_TYPE: maria
    depends_on:
      - db

  db:
    container_name: db
    image: mariadb:10.11
    restart: always
    ports:
      - "3306:3306"
    environment:
      DB_USER: test
      DB_PASS: P@ssw0rd
      DB_NAME: testdb
      MARIADB_ROOT_PASSWORD: P@ssw0rd
```



```
GNU nano 8.0
services:
  testapp:
    container_name: testapp
    image: site:latest
    restart: always
    ports:
      - "8080:8000"
    environment:
      DB_HOST: "192.168.30.2"
      DB_PORT: "3306"
      DB_NAME: testdb
      DB_USER: test
      DB_PASS: P@ssw0rd
      DB_TYPE: maria
    depends_on:
      - db

  db:
    container_name: db
    image: mariadb:10.11
    restart: always
    ports:
      - "3306:3306"
    environment:
      DB_USER: test
      DB_PASS: P@ssw0rd
      DB_NAME: testdb
      MARIADB_ROOT_PASSWORD: P@ssw0rd
```

Рисунок 39

Параметры в файле web.yml

```

services:                                # блок сервисов (контейнеры docker)
                                           # ниже контейнеры которые поднимаем

testapp:                                # название сервиса
  container_name: testapp                # объявляем имя контейнера
  image: site:latest                     # указываем имя используемого Docker-
                                           # образа
  restart: always                         # опция перезагрузки (автоматически
                                           # перезапускаться всегда, независимо от
                                           # причины его остановки)
  ports:                                # проброс портов внешний (8080) на
    - "8080:8000"                           внутренний (8000)
  environment:                          # список переменных окружения, которые
                                           # пробрасываются внутрь контейнера;
                                           # описаны в файле /docker/readme.txt
                                           # на Additional.iso

  DB_HOST: "192.168.30.2"                 # хост БД (IPv4 BR-SRV)
  DB_PORT: "3306"                       # порт БД (используем стандартный порт
                                           # для соединений с базами данных MySQL
                                           # имя базы данных
  DB_NAME: testdb                         # пользователь БД
  DB_USER: test                           # пароль БД
  DB_PASS: P@ssw0rd                       # тип БД
  DB_TYPE: maria                         # зависимости запуска. перед запуском
depends_on:                               контейнера testapp запустить контейнер
  - db                                       db
                                           #
db:                                       # название сервиса
  container_name: db                     # объявляем имя контейнера
  image: mariadb:10.11                   # имя используемого Docker-образа
  restart: always                         # опция перезагрузки (автоматически
                                           # перезапускаться всегда, независимо от
                                           # причины его остановки)
  ports:                                # проброс портов внешний (3306) на
    - "3306:3306"                           внутренний (3306)
  environment:                          # список переменных окружения, которые
                                           # пробрасываются внутрь контейнера;
                                           # описаны в файле /docker/readme.txt
                                           # на Additional.iso
                                           # пользователь БД
  DB_USER: test                           # пароль БД
  DB_PASS: P@ssw0rd                       # имя базы данных
  DB_NAME: testdb                         # переменная в docker-compose для
MARIADB_ROOT_PASSWORD:                 контейнера MariaDB, которая
P@ssw0rd                                     устанавливает пароль пользователя root.

```

Запускаем контейнеры:

```
# docker-compose -f web.yml up -d
```

Смотрим список запущенных контейнеров:

```
# docker ps
```

Видим что контейнер *site:latest* перезапускается (не может подключиться к контейнеру *mariadb:10.11* (в контейнере отсутствует база данных))

6.3 Создание базы данных в контейнере

Заходим в контейнер:

```
# docker exec -it db mysql -u root -p
```

Вводим пароль пользователя *root* (*P@ssw0rd*) (задан в параметре *MARIADB_ROOT_PASSWORD* в *web.yaml*)

Создам базу данных:

```
CREATE DATABASE testdb;
```

Создаем пользователя:

```
CREATE USER 'test'@'%' IDENTIFIED BY 'P@ssw0rd';
```

Даем права пользователю *test* на БД *testdb*

```
GRANT ALL PRIVILEGES ON testdb.* TO 'test'@'%';
```

Обновляем привилегии:

```
FLUSH PRIVILEGES;
```

Выходим:

```
\q
```

Останавливаем контейнеры:

```
# docker-compose -f web.yaml stop
```

Запускаем контейнеры:

```
# docker-compose -f web.yaml up -d
```

Смотрим список запущенных контейнеров:

```
# docker ps
```

```
lroot@br-srv ~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
221e3fa874b5   site:latest    "sh -c 'python3 -m ap"   14 hours ago   Up 2 hours   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp   testapp
eba567e5b85e   mariadb:10.11  "docker-entrypoint.sq"   14 hours ago   Up 2 hours   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   db
lroot@br-srv ~$
```

Рисунок 40

Если все *OK* переходим на *HQ-CLI* и в браузере вводим IP BR-SRV и порт 8080

`http://192.168.30.2:8080`

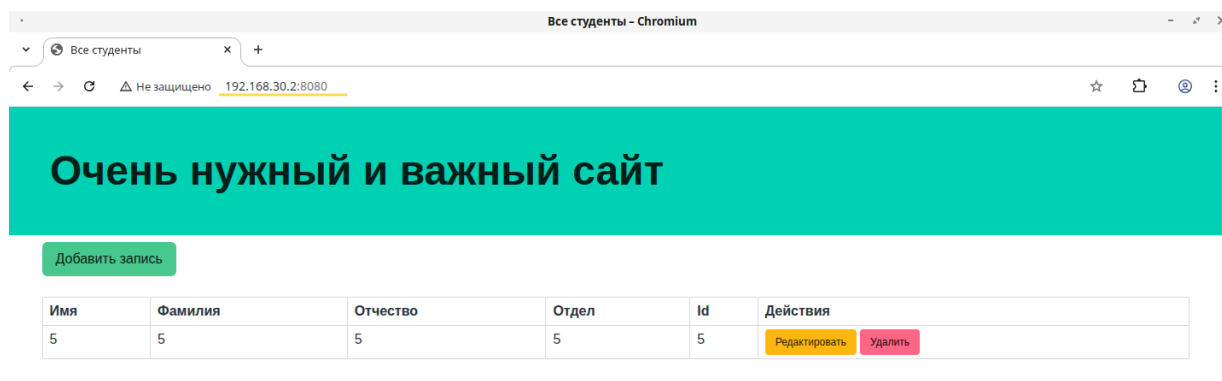


Рисунок 41

7. Веб приложение на HQ-SRV

Задание

Разверните веб приложение на сервере HQ-SRV:

- Используйте веб-сервер `apache`
- В качестве системы управления базами данных используйте `mariadb`
- Файлы веб приложения и дамп базы данных находятся в директории `web` образа `Additional.iso`
- Выполните импорт схемы и данных из файла `dump.sql` в базу данных `webdb`
- Создайте пользователя `web` с паролем `P@ssw0rd` и предоставьте ему права доступа к этой базе данных
- Файлы `index.php` и директорию `images` скопируйте в каталог веб сервера `apache`
- В файле `index.php` укажите правильные учётные данные для подключения к БД
- Запустите веб сервер и убедитесь в работоспособности приложения
- Основные параметры отметьте в отчёте

Реализация

Переводим `selinux` в режим уведомлений (см. Приложение).

Установка и настройка веб-сервера Apache

Устанавливаем веб-сервер Apache:

```
# dnf install httpd
```

Запускаем службу `httpd` и добавляем ее в автозагрузку:

```
# systemctl enable httpd --now
```

Устанавливаем `php` и дополнительные расширения

```
# dnf install php php-mysqlnd
```

Установка и настройка MySQL

Устанавливаем СУБД *MariaDB*

```
# dnf install mariadb-server mariadb
```

Запускаем службу *mysql* и добавляем ее в автозагрузку:

```
# systemctl enable mariadb --now
```

Запускаем встроенный инструмент настройки *MariaDB* и производим настройку:

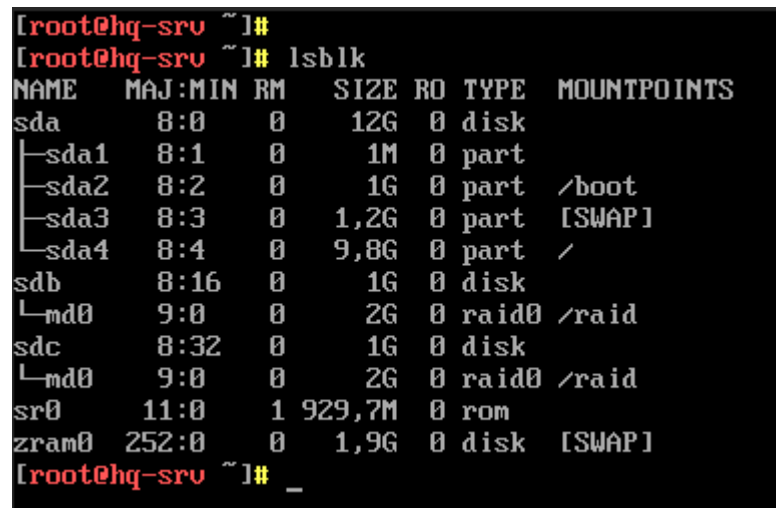
```
# mysql_secure_installation
```

Устанавливаем пароль *root* для *mariadb*. Во всех пунктах вводим *Y* (Yes)

Монтирование Additional.iso

Смотрим подключен ли *cd-rom*

```
# lsblk
```



```
[root@hq-srv ~]#  
[root@hq-srv ~]# lsblk  
NAME        MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS  
sda          8:0    0   12G  0 disk  
├─sda1       8:1    0    1M  0 part  
├─sda2       8:2    0    1G  0 part  /boot  
├─sda3       8:3    0   1,2G  0 part  [SWAP]  
└─sda4       8:4    0   9,8G  0 part  /  
sdb          8:16   0    1G  0 disk  
└─md0        9:0    0    2G  0 raid0 /raid  
sdc          8:32   0    1G  0 disk  
└─md0        9:0    0    2G  0 raid0 /raid  
sr0         11:0    1 929,7M  0 rom  
zram0       252:0    0    1,9G  0 disk  [SWAP]  
[root@hq-srv ~]# _
```

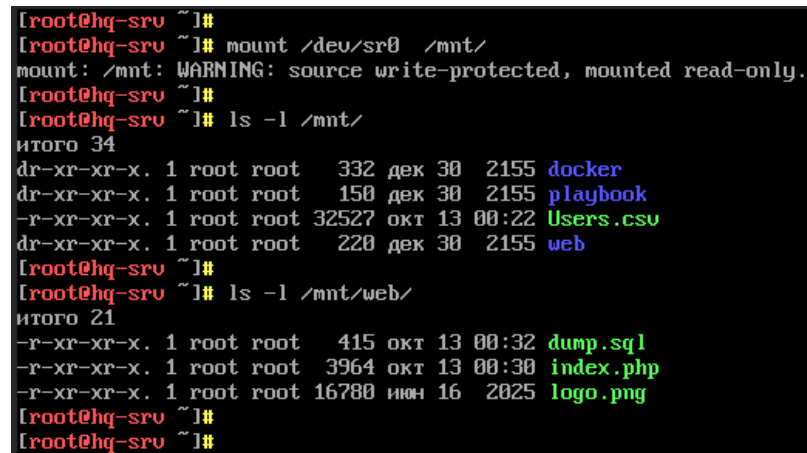
Рисунок 42

Монтируем *Additional.iso*

```
# mount /dev/sr0 /mnt/
```


Смотрим содержимое Additional.iso

```
# ls -l /mnt/  
# ls -l /mnt/web
```



```
[root@hq-srv ~]#  
[root@hq-srv ~]# mount /dev/sr0 /mnt/  
mount: /mnt: WARNING: source write-protected, mounted read-only.  
[root@hq-srv ~]#  
[root@hq-srv ~]# ls -l /mnt/  
итого 34  
dr-xr-xr-x. 1 root root 332 дек 30 2155 docker  
dr-xr-xr-x. 1 root root 150 дек 30 2155 playbook  
-r-xr-xr-x. 1 root root 32527 окт 13 00:22 Users.csv  
dr-xr-xr-x. 1 root root 220 дек 30 2155 web  
[root@hq-srv ~]#  
[root@hq-srv ~]# ls -l /mnt/web/  
итого 21  
-r-xr-xr-x. 1 root root 415 окт 13 00:32 dump.sql  
-r-xr-xr-x. 1 root root 3964 окт 13 00:30 index.php  
-r-xr-xr-x. 1 root root 16780 июн 16 2025 logo.png  
[root@hq-srv ~]#  
[root@hq-srv ~]#
```

Рисунок 43

Создание базы данных и импорт данных

Подключаемся к *mariadb*

```
# mysql -u root -p
```

Создаем базу данных:

```
CREATE DATABASE webdb;
```

Создаем пользователя:

```
CREATE USER 'web'@'%' IDENTIFIED BY 'P@ssw0rd';
```

Задаем права пользователя на БД:

```
GRANT ALL PRIVILEGES ON webdb.* TO 'web'@'%' ;
```

Перечитываем привилегии:

```
FLUSH PRIVILEGES;
```

Выходим:

```
quit;
```

Импортируем схему и данные из файла *dump.sql* в базу данных *webdb*:

```
# mysql webdb < /mnt/web/dump.sql
```

Импорт данных веб приложения

Из директории */mnt/web/* копируем файл *index.php* и *logo.png* в */var/www/html/*

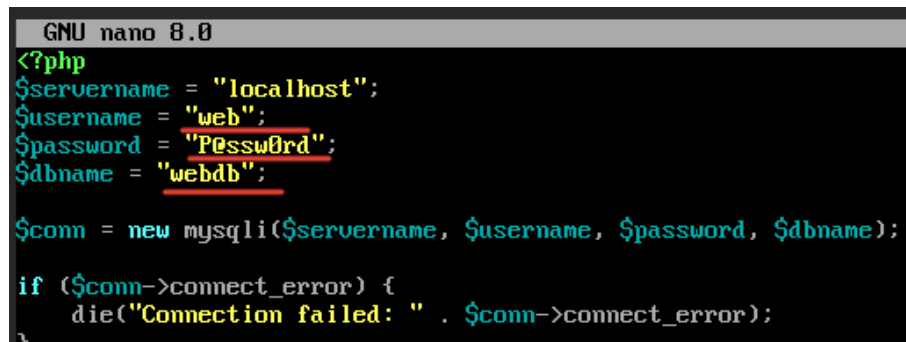
```
# cp /mnt/web/index.php /var/www/html/  
# cp /mnt/web/logo.png /var/www/html/
```

В файле */var/www/html/index.php* необходимо указать учётные данные для подключения к БД:

Открываем файл

```
# nano /var/www/html/index.php
```

Вносим изменения:



```
GNU nano 8.0  
<?php  
$servername = "localhost";  
$username = "web";  
$password = "P@ssw0rd";  
$dbname = "webdb";  
  
$conn = new mysqli($servername, $username, $password, $dbname);  
  
if ($conn->connect_error) {  
    die("Connection failed: " . $conn->connect_error);  
}
```

Рисунок 44

Перезапускаем службу *httpd*

```
# systemctl restart httpd
```

Проверка

Проверяем доступ до веб приложения введя на HQ-CLI IP-адрес HQ-SRV- *http://192.168.100.2*

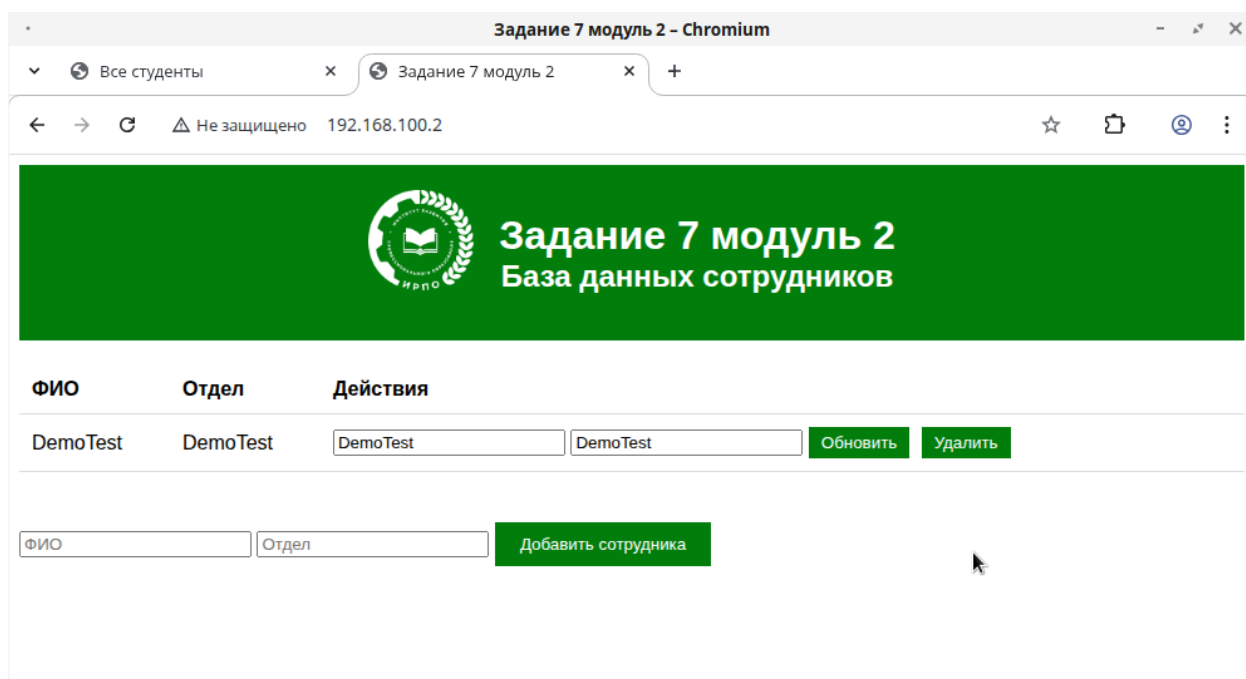


Рисунок 45

8. Статическая трансляция портов

Задание

На маршрутизаторах сконфигурируйте статическую трансляцию портов:

- Пробросьте порт 8080 в порт приложения testapp BR-SRV на маршрутизаторе BR-RTR, для обеспечения работы приложения testapp извне
- Пробросьте порт 8080 в порт веб приложения на HQ-SRV на маршрутизаторе HQ-RTR, для обеспечения работы веб приложения извне
- Пробросьте порт 2026 на маршрутизаторе HQ-RTR в порт 2026 сервера HQ-SRV, для подключения к серверу по протоколу ssh из внешних сетей
- Пробросьте порт 2026 на маршрутизаторе BR-RTR в порт 2026 сервера BR-SRV, для подключения к серверу по протоколу ssh из внешних сетей.

Реализация

8.1 Проброс порта 8080 в порт приложения testapp BR-SRV на маршрутизаторе BR-RTR

Создаем правило *nftables* на BR-RTR, которое будет перенаправлять внешние подключения к *BR-RTR* на порту *8080* -> на порт *8080* сервера *BR-SRV*.

Добавим цепочку *prerouting* в таблицу *nat* в ранее созданный файл (модуль 1) с правилами *nftables* */etc/nftable/br-rtr.nft*

Открываем файл на BR-RTR

```
# nano etc/nftable/br-rtr.nft
```

И дописываем правила в начало таблицы

```
chain PREROUTING {  
    type nat hook prerouting priority filter;  
    ip daddr 172.16.2.2 tcp dport 8080 dnat ip to 192.168.30.2:8080  
}
```

```
GNU nano 8.0 /etc
table inet nat {
  chain PREROUTING {
    type nat hook prerouting priority filter;
    ip daddr 172.16.2.2 tcp dport 8080 dnat ip to 192.168.30.2:8080
  }
  chain POSTROUTING {
    type nat hook postrouting priority srcnat;
    oifname "ens18" masquerade
  }
}
```

Рисунок 46

Проверяем возможность доступа к ресурсу извне. Введя на HQ-CLI IP-адрес BR-RTR смотрящий в сторону ISP и порт 8080. (172.16.2.2:8080)

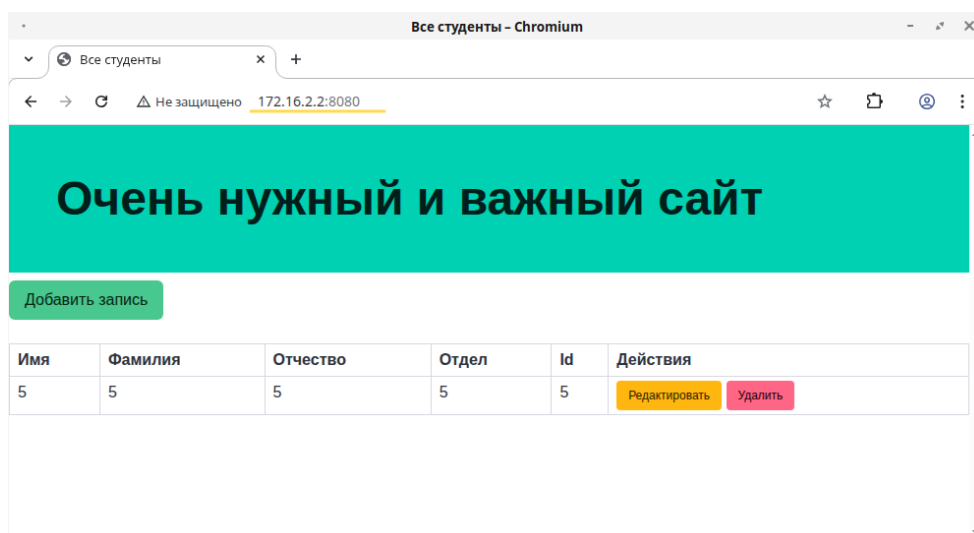


Рисунок 47

8.2 Проброс порта 8080 в порт веб приложения на HQ-SRV на маршрутизаторе HQ-RTR

Создаем правило *nftables* на HQ-RTR, которое будет перенаправлять внешние подключения к HQ-RTR на порту 8080 -> на порт 80 сервера HQ-SRV.

Добавим цепочку *prerouting* в таблицу *nat* в ранее созданный файл с правилами *nftables* `/etc/nftable/hq-rtr.nft`

Открываем файл на HQ-RTR

```
# nano etc/nftable/hq-rtr.nft
```

И дописываем правила в начало таблицы:

```
chain PREROUTING {  
    type nat hook prerouting priority filter;  
    ip daddr 172.16.1.2 tcp dport 8080 dnat ip to 192.168.100.2:80  
}
```

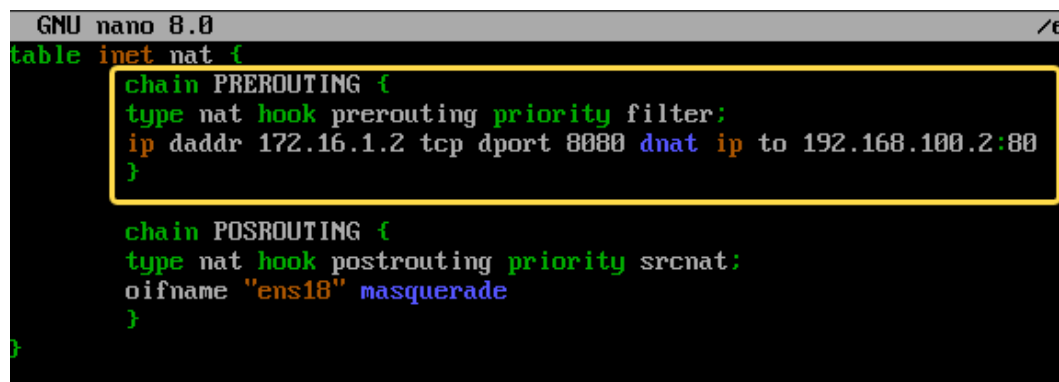


Рисунок 48

Проверяем возможность доступа к ресурсу извне. Введя на HQ-CLI IP-адрес HQ-RTR смотрящий в сторону ISP и порт 8080. (172.16.1.2:8080)

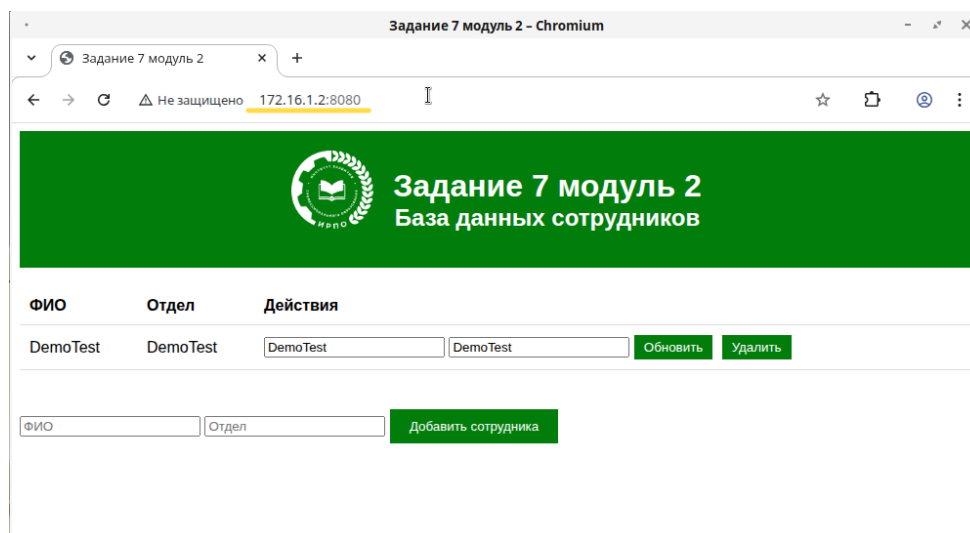


Рисунок 49

8.3 Проброс порта 2026 на маршрутизаторе HQ-RTR в порт 2026 сервера HQ-SRV

Добавляем правило в *nftables* на HQ-RTR, которое будет перенаправлять внешние подключения к HQ-RTR на порту 2026 -> на порт 2026 сервера HQ-SRV.

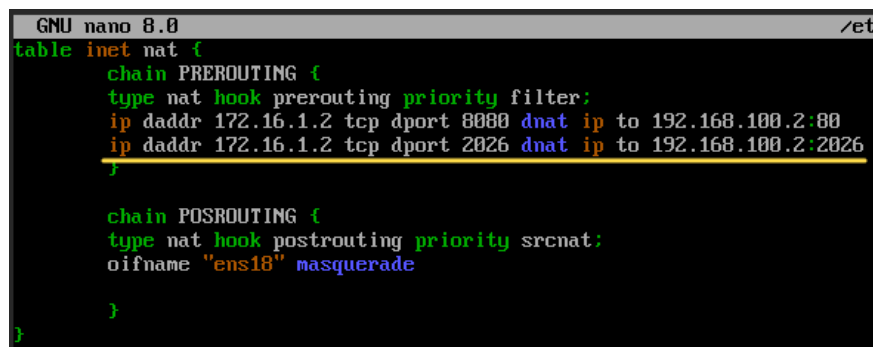
Добавим правило в цепочку *prerouting* в ранее созданный файл с правилами *nftables* `/etc/nftable/hq-rtr.nft`

Открываем файл на HQ-RTR:

```
# nano /etc/nftable/hq-rtr.nft
```

И дописываем правило в цепочку *chain PREROUTING*:

```
ip daddr 172.16.1.2 tcp dport 2026 dnat ip to 192.168.100.2:2026
```



```
GNU nano 8.8 /et
table inet nat {
  chain PREROUTING {
    type nat hook prerouting priority filter;
    ip daddr 172.16.1.2 tcp dport 8080 dnat ip to 192.168.100.2:80
    ip daddr 172.16.1.2 tcp dport 2026 dnat ip to 192.168.100.2:2026
  }

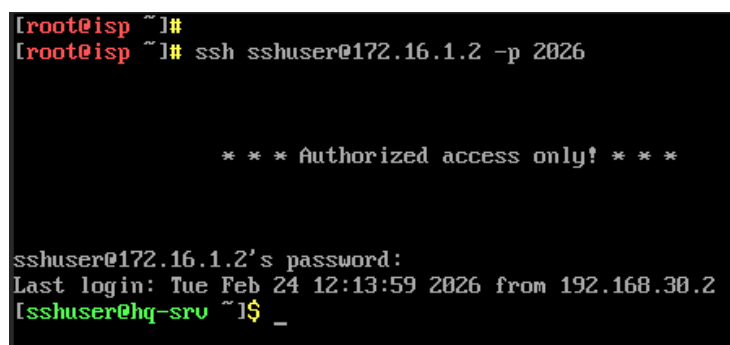
  chain POSTROUTING {
    type nat hook postrouting priority srcnat;
    oifname "ens18" masquerade
  }
}
```

Рисунок 50

Проверяем

Подключаемся по SSH с ISP к HQ-SRV

```
# ssh sshuser@172.16.1.2 -p 2026
```



```
[root@isp ~]#
[root@isp ~]# ssh sshuser@172.16.1.2 -p 2026

* * * Authorized access only! * * *

sshuser@172.16.1.2's password:
Last login: Tue Feb 24 12:13:59 2026 from 192.168.30.2
[sshuser@hq-srv ~]$_
```

Рисунок 51

8.4 Проброс порта 2026 на маршрутизаторе BR-RTR в порт 2026 сервера BR-SRV

Добавляем правило в *nftables* на BR-RTR, которое будет перенаправлять внешние подключения к *BR-RTR* на порту *2026* -> на порт *2026* сервера *BR-SRV*.

Добавим правило в цепочку *prerouting* в ранее созданный файл с правилами *nftables* `/etc/nftable/br-rtr.nft`

Открываем файл на BR-RTR:

```
# nano /etc/nftable/br-rtr.nft
```

И дописываем правило в цепочку *chain PREROUTING*:

```
ip daddr 172.16.2.2 tcp dport 2026 dnat ip to 192.168.30.2:2026
```



Рисунок 52

Проверяем

Подключаемся по SSH с ISP к BR-SRV

```
# ssh sshuser@172.16.2.2 -p 2026
```

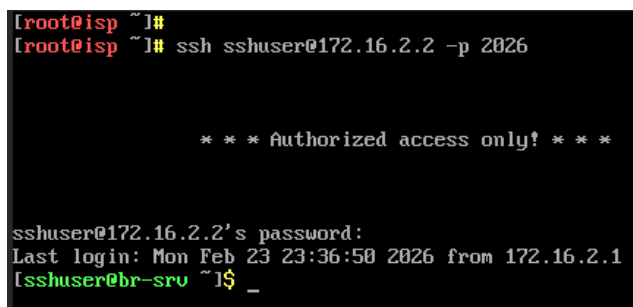


Рисунок 53

9. Обратный прокси-сервер на ISP

Задание

ISP **Настройте веб-сервер nginx как обратный прокси-сервер на**

- При обращении по доменному имени web.au-team.irpo у клиента должно открываться веб приложение на HQ-SRV
- При обращении по доменному имени docker.au-team.irpo клиента должно открываться веб приложение testapp

Реализация

Переводим *selinux* в режим уведомлений (см. Приложение).

Устанавливаем *Nginx*

```
# dnf install nginx -y
```

Запускаем и добавляем в автозагрузку

```
# systemctl enable --now nginx
```

Открываем на редактирование конфигурационный файл *Nginx*

```
# nano /etc/nginx/nginx.conf
```

Спускаемся в конец документа и перед *последней фигурной скобкой* прописываем:

```
server {
    listen 80;
    server_name web.au-team.irpo;

    location / {
        proxy_pass http://172.16.1.2:8080;
    }
}
server {
    listen 80;
    server_name docker.au-team.irpo;

    location / {
        proxy_pass http://172.16.2.2:8080;
    }
}
```

```
server {
    listen 80;
    server_name web.au-team.irpo;

    location / {
        proxy_pass http://172.16.1.2:8080;
    }
}

server {
    listen 80;
    server_name docker.au-team.irpo;

    location / {
        proxy_pass http://172.16.2.2:8080;
    }
}
```

Рисунок 54

Проверяем синтаксис конфигурационного файла *Nginx*

```
# nginx -t
```

Перезапускаем *nginx*

```
# systemctl restart nginx
```

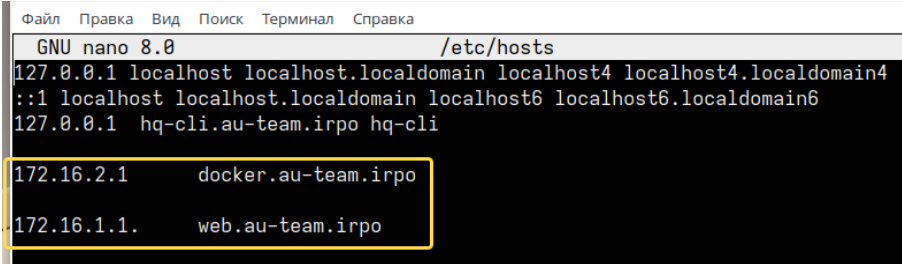
В домене *SambaDC* на HQ-SRV DNS записям *web.au-team.irpo* и *docker.au-team.irpo* соответствуют другие IP адреса (согласно 1 модуля). Внесем дополнительные записи в */etc/hosts* на HQ-CLI

Открываем файл *hosts* на HQ-CLI:

```
# nano /etc/hosts
```

Добавляем записи

```
172.16.2.1    docker.au-team.irpo
172.16.1.1    web.au-team.irpo
```



```
Файл  Правка  Вид  Поиск  Терминал  Справка
GNU nano 8.0 /etc/hosts
127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6
127.0.0.1 hq-cli.au-team.irpo hq-cli
172.16.2.1    docker.au-team.irpo
172.16.1.1    web.au-team.irpo
```

Рисунок 55

Проверка

На HQ-CLI в браузере вводим `docker.au-team.irpo` и `web.au-team.irpo`

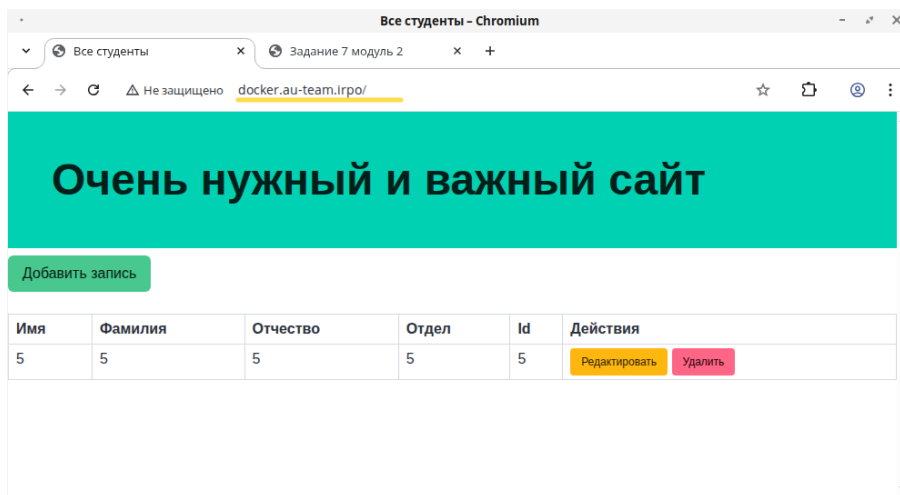


Рисунок 56

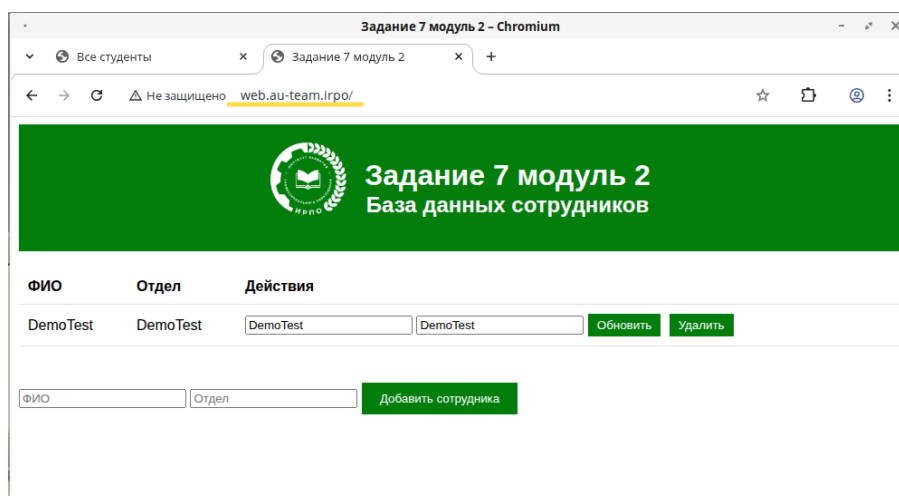


Рисунок 57

10. Web-based аутентификация

Задание

На маршрутизаторе ISP настройте web-based аутентификацию:

- При обращении к сайту `web.au-team.irpo` клиенту должно быть предложено ввести аутентификационные данные
- В качестве логина для аутентификации выберите WEB с паролем `P@ssw0rd`
- Выберите файл `/etc/nginx/.htpasswd` в качестве хранилища учётных записей
- При успешной аутентификации клиент должен перейти на веб сайт.

Реализация

`Htpasswd` — утилита для работы с файлом паролей, обычно входящая в состав дистрибутива веб-сервера Apache. С её помощью создают файлы паролей, добавляют новые записи или модифицируют существующие.

Файл `.htpasswd` содержит пароли для доступа к ресурсу. Он состоит из строк, каждая из которых соответствует паре `логин/хеш пароля`. Логин отделяется от хеша двоеточием.

Устанавливаем `htpasswd` который входит в пакет `httpd-tools`:

```
# dnf install httpd-tools
```

Проверяем, что `htpasswd` установлен:

```
# htpasswd -h
```

При помощи утилиты `htpasswd` создаем пользователя `WEB` с паролем `P@ssw0rd` и добавляем информацию о нём в файл `/etc/nginx/.htpasswd`

```
# htpasswd -c /etc/nginx/.htpasswd WEB
```

В конфигурационном файле `/etc/nginx/nginx.conf` добавляем `web-based` аутентификацию для доступа к сайту `web.au-team.irpo`

```
# nano /etc/nginx/nginx.conf
```

Дописываем следующие строки в блок `server` проксирование `web.au-team.irpo`)

```
auth_basic "Restricted area";  
auth_basic_user_file /etc/nginx/.htpasswd;
```

```
server {  
    listen 80;  
    server_name web.au-team.irpo;  
  
    location / {  
        proxy_pass http://172.16.1.2:8080;  
        auth_basic "Restricted area";  
        auth_basic_user_file /etc/nginx/.htpasswd;  
    }  
}
```

Рисунок 58

Перезапускаем `nginx`

```
# systemctl restart nginx
```

Проверка

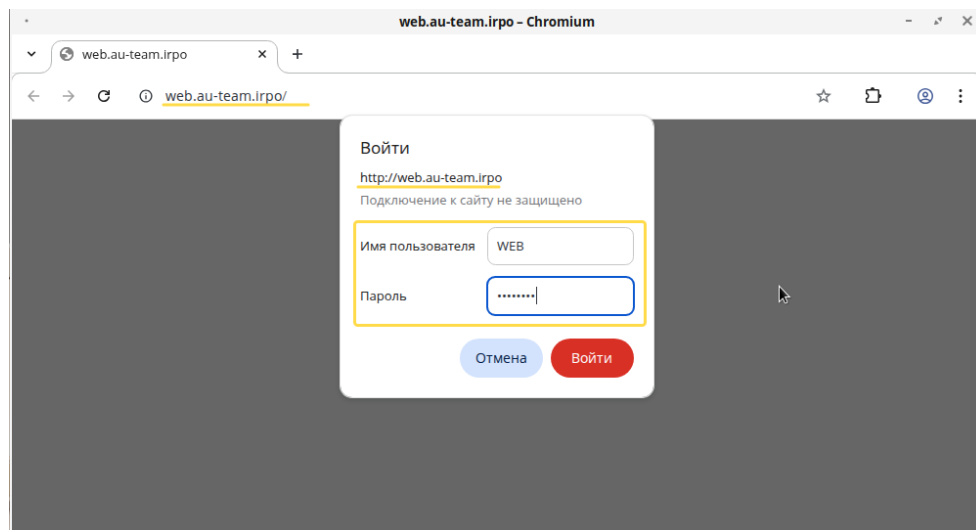


Рисунок 59

11. Установка Яндекс Браузер

Задание

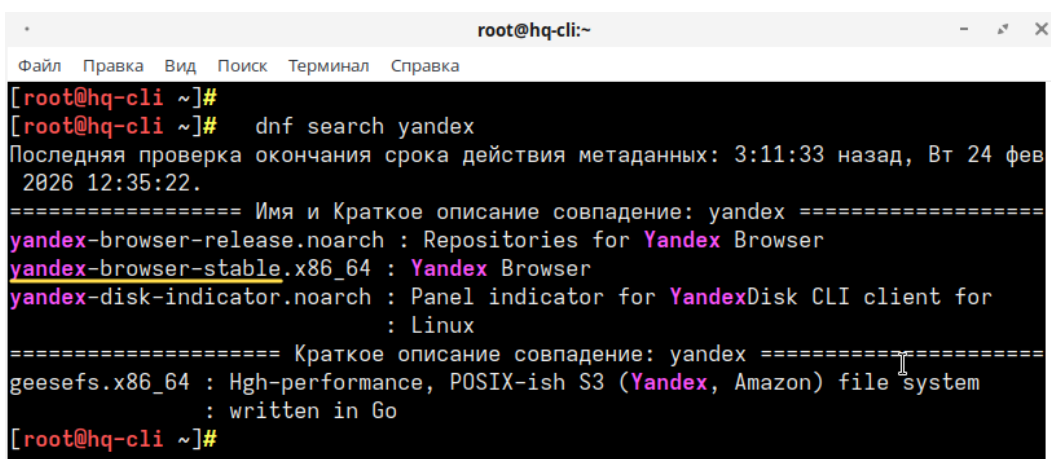
Удобным способом установите приложение Яндекс Браузер на HQ-CLI

- Установку браузера отметьте в отчёте.

Реализация

Ищем название пакета Яндекс браузера

```
# dnf search yandex
```



```
root@hq-cli:~  
Файл Правка Вид Поиск Терминал Справка  
[root@hq-cli ~]#  
[root@hq-cli ~]# dnf search yandex  
Последняя проверка окончания срока действия метаданных: 3:11:33 назад, Вт 24 фев 2026 12:35:22.  
===== Имя и Краткое описание совпадение: yandex =====  
yandex-browser-release.noarch : Repositories for Yandex Browser  
yandex-browser-stable.x86_64 : Yandex Browser  
yandex-disk-indicator.noarch : Panel indicator for YandexDisk CLI client for  
                               : Linux  
===== Краткое описание совпадение: yandex =====  
geeseefs.x86_64 : Hgh-performance, POSIX-ish S3 (Yandex, Amazon) file system  
                  : written in Go  
[root@hq-cli ~]#
```

Рисунок 60

Устанавливаем Яндекс браузер:

```
# dnf install yandex-browser-stable -y
```

После установки Яндекс.Браузер доступен «Главного меню» -> «Интернет» -> «Yandex Browser».

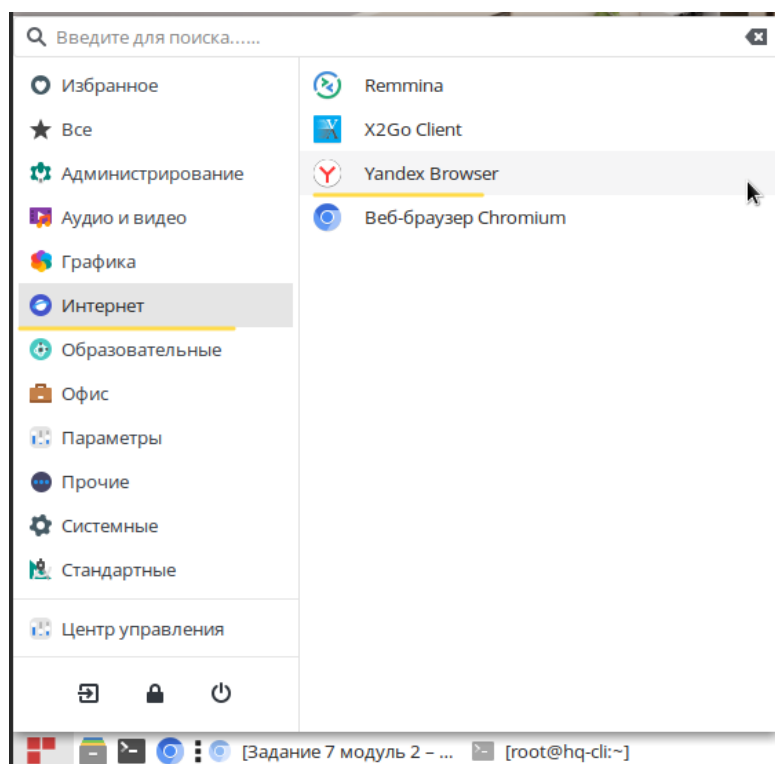


Рисунок 61

Приложение

Перевод SELinux в режим уведомлений.

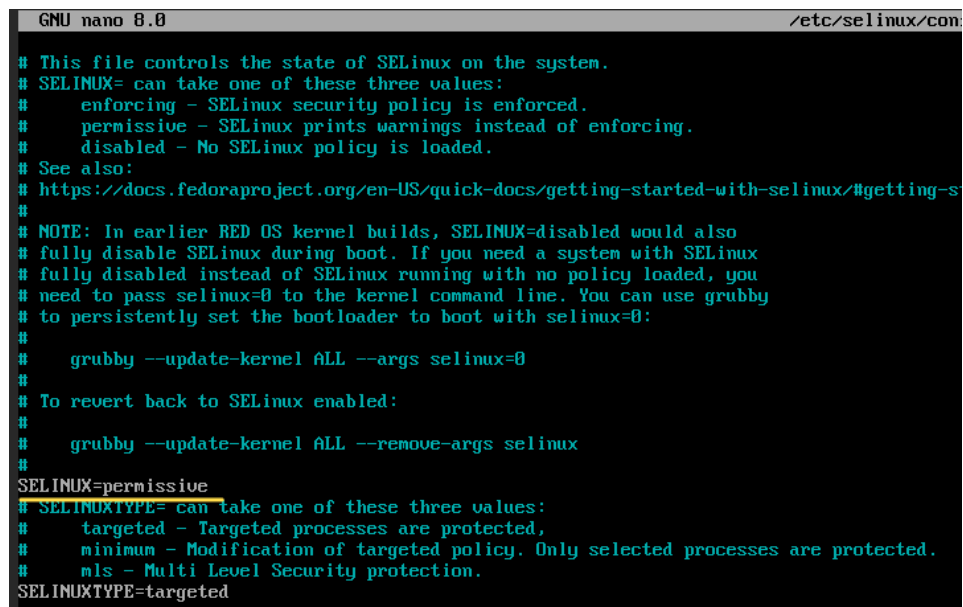
Переводим SELinux в режим уведомлений. Для этого изменяем содержимое конфигурационного файла:

```
# nano /etc/selinux/config
```

Заменяем текст SELINUX=enforcing на SELINUX=permissive.

Затем выполняем (применение настроек без перезагрузки)

```
# setenforce 0
```



```
GNU nano 8.0 /etc/selinux/con
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
#   enforcing - SELinux security policy is enforced.
#   permissive - SELinux prints warnings instead of enforcing.
#   disabled - No SELinux policy is loaded.
# See also:
# https://docs.fedoraproject.org/en-US/quick-docs/getting-started-with-selinux/#getting-s
#
# NOTE: In earlier RED OS kernel builds, SELINUX=disabled would also
# fully disable SELinux during boot. If you need a system with SELinux
# fully disabled instead of SELinux running with no policy loaded, you
# need to pass selinux=0 to the kernel command line. You can use grubby
# to persistently set the bootloader to boot with selinux=0:
#
#   grubby --update-kernel ALL --args selinux=0
#
# To revert back to SELinux enabled:
#
#   grubby --update-kernel ALL --remove-args selinux
#
SELINUX=permissive
# SELINUXTYPE= can take one of these three values:
#   targeted - Targeted processes are protected,
#   minimum - Modification of targeted policy. Only selected processes are protected.
#   mls - Multi Level Security protection.
SELINUXTYPE=targeted
```

Рисунок 62